

Single-Cell Multi-Omics in R

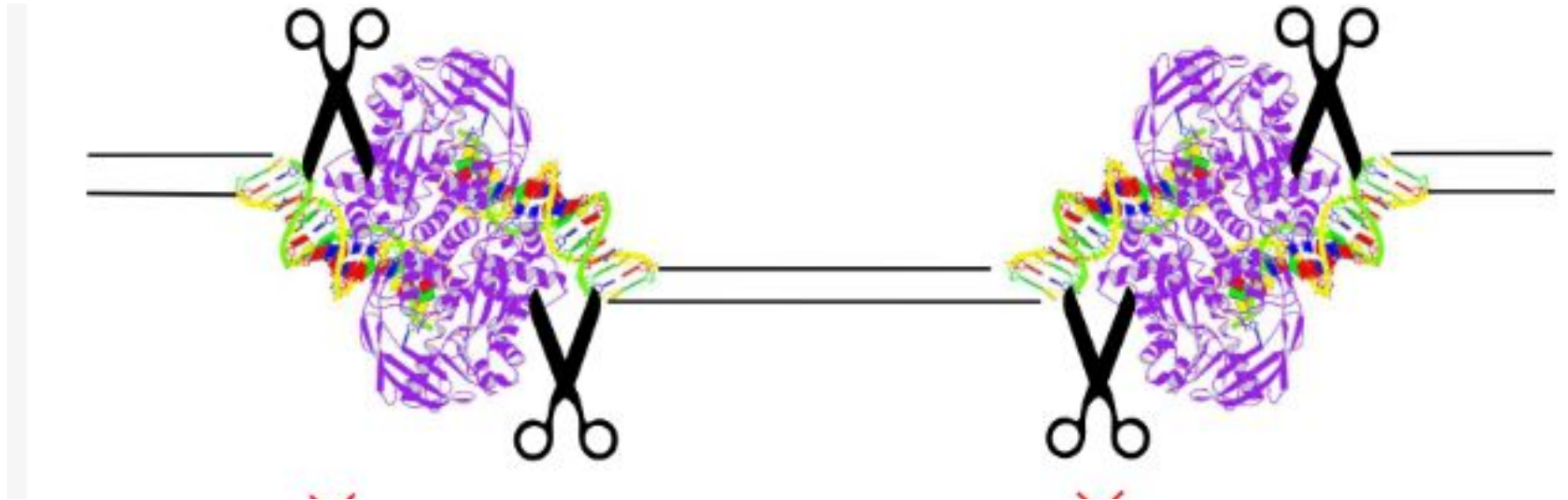
April 28, 2026

Steve Rozen

severozen [at] pm.me

sr110 [at] duke.edu

See also https://github.com/steverozen/cite_and_atac_wkshp



Workshop objectives:

More experience with RStudio / R package / R environment management

Examining R data structures

Understand Seurat ecosystem as applied to CITE-seq and ATAC-seq analysis

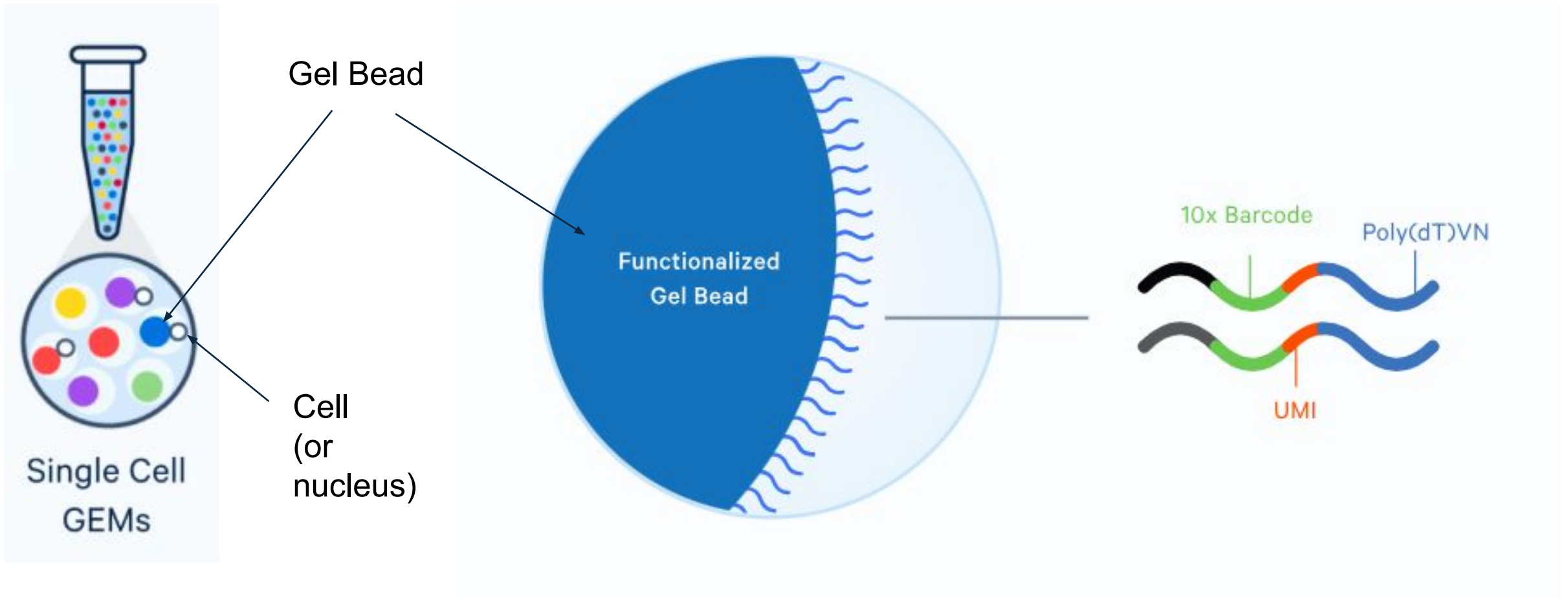
CITE-seq and single-cell ATAC-seq basic concepts

1 CITE-seq: https://satijalab.org/seurat/articles/multimodal_vignette, 2023-11-16 Cord PBMCs, 11 surface proteins

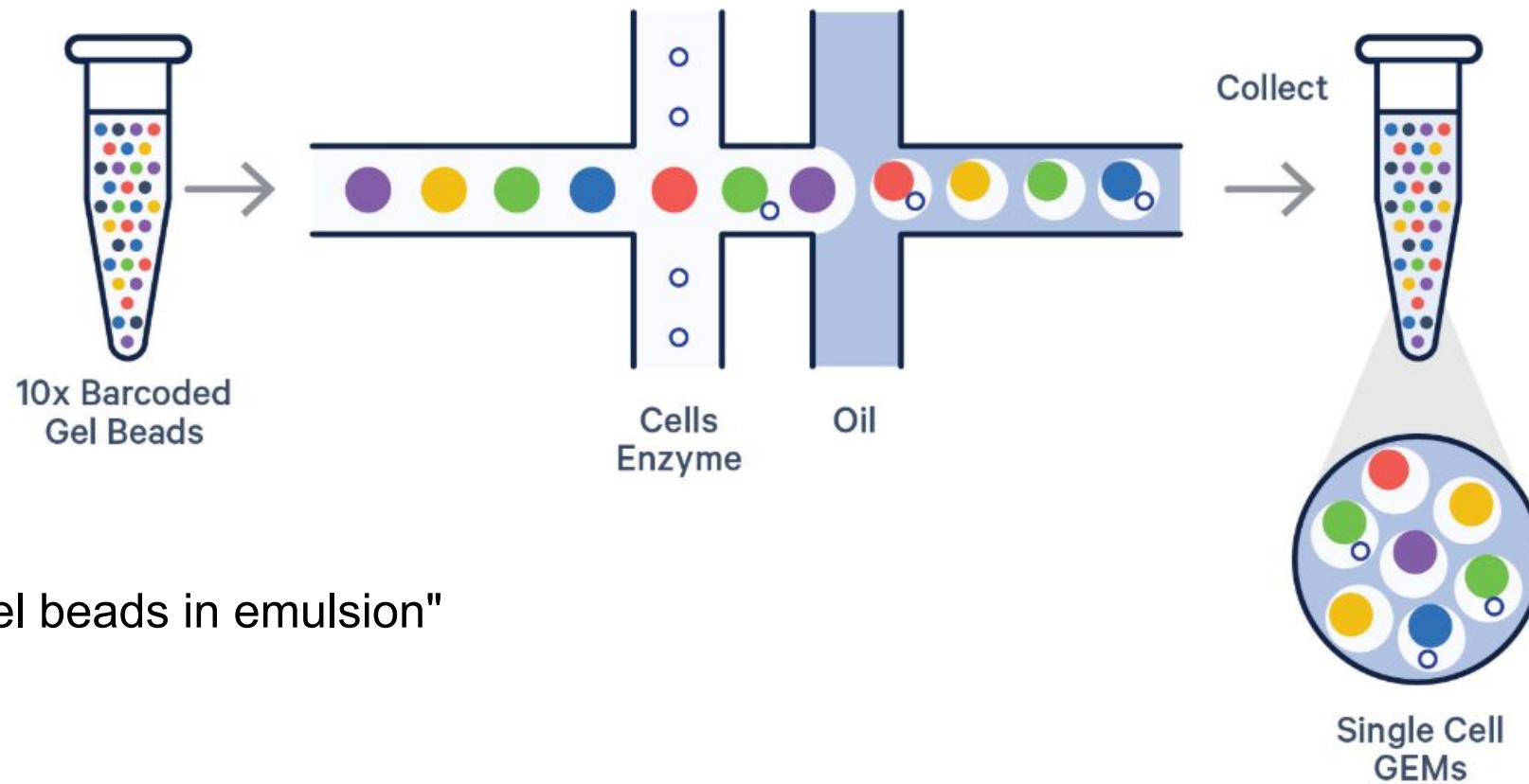
2 ATAC seq / Signac PBMC ATAC seq – 2024-08-24
https://stuartlab.org/signac/articles/pbmc_vignette.html

For information on R libraries and data needed for the tutorials see:

1 CITE-seq



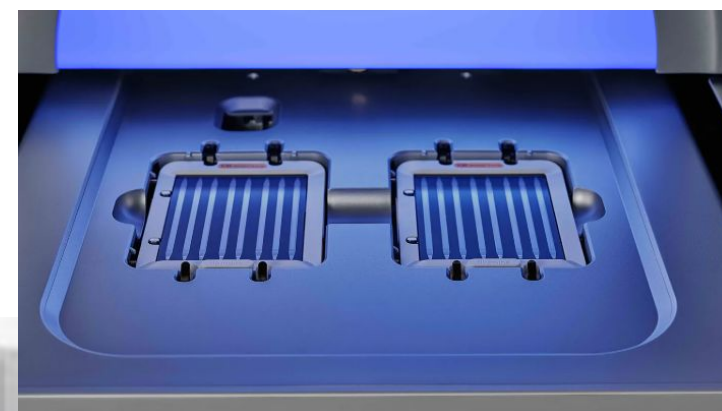
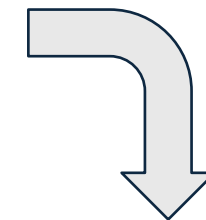
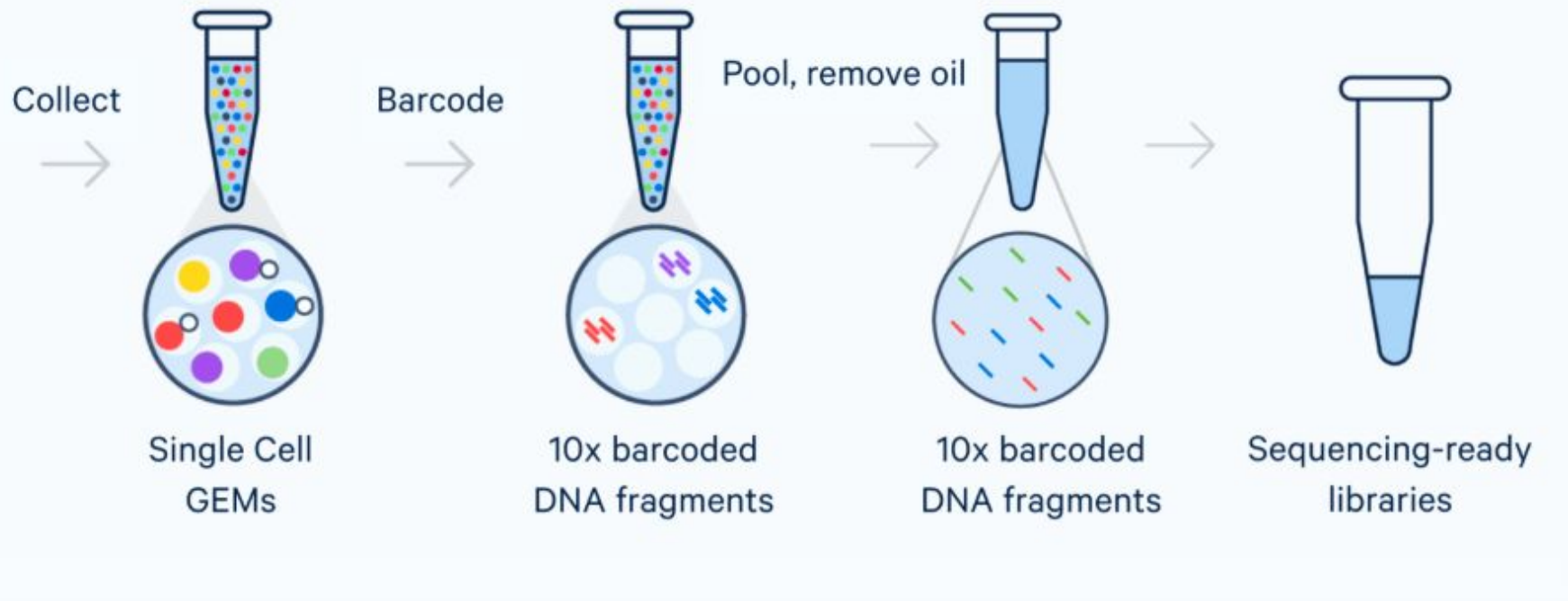
<https://www.10xgenomics.com/platforms/chromium/technology>



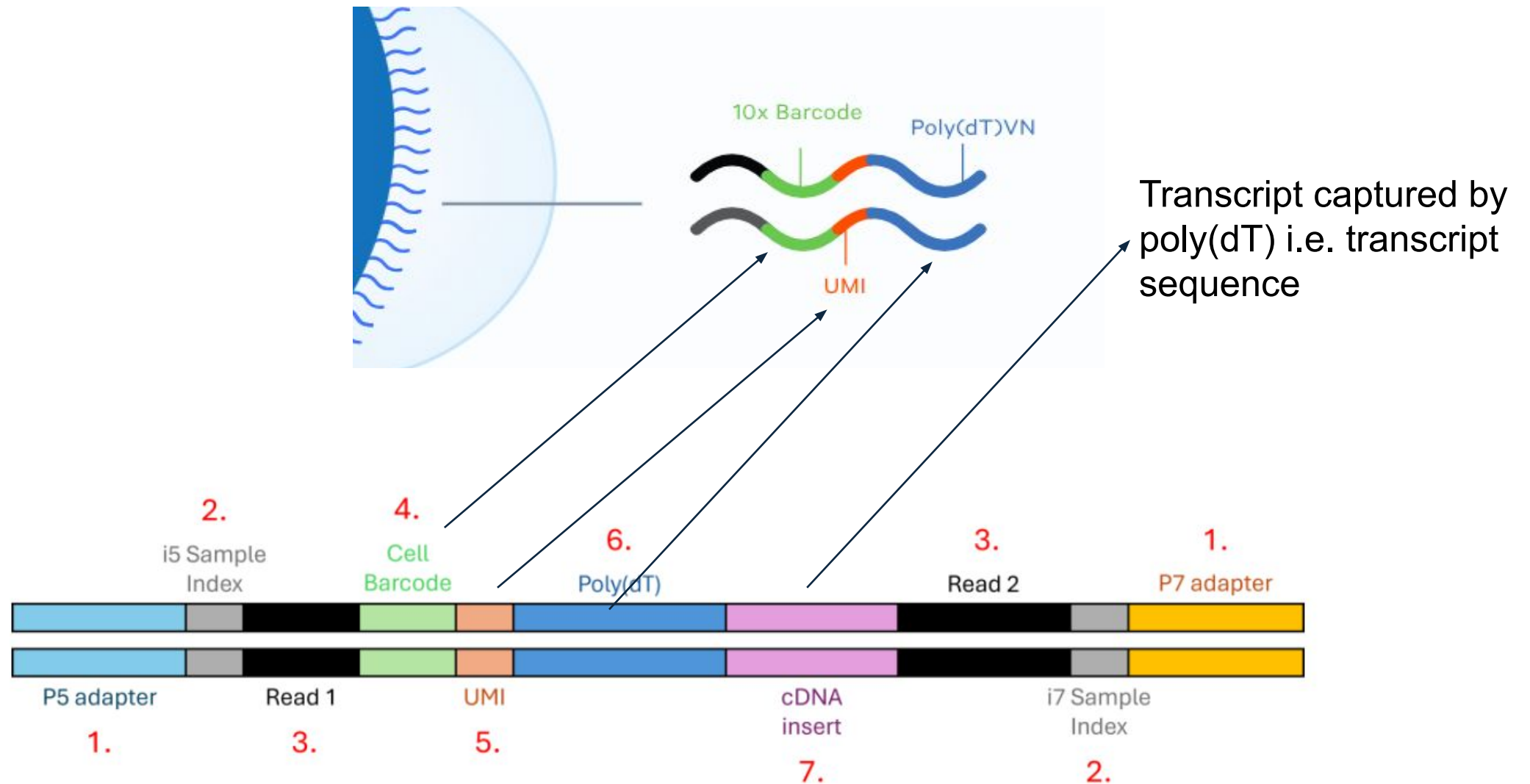
GEM = "Gel beads in emulsion"

https://cdn.10xgenomics.com/image/upload/f_auto,q_auto,w_900,dpr_2.0/v1709753672/blog/GEM-X%20Launch%20blog/vertical_flipped_GEM96_H1975_high_4_S0001.gif

<https://www.10xgenomics.com/blog/the-next-generation-of-single-cell-rna-seq-an-introduction-to-gem-x-technology>



Short-read
sequencer



Each UMI is 1 input cDNA molecule
 Each Cell Barcode is 1 cell/nucleus from a droplet

The sequencer generates "BCL" files

Downstream processing uses "fastq" files

For single cell RNA seq we often used 10X's cellranger software

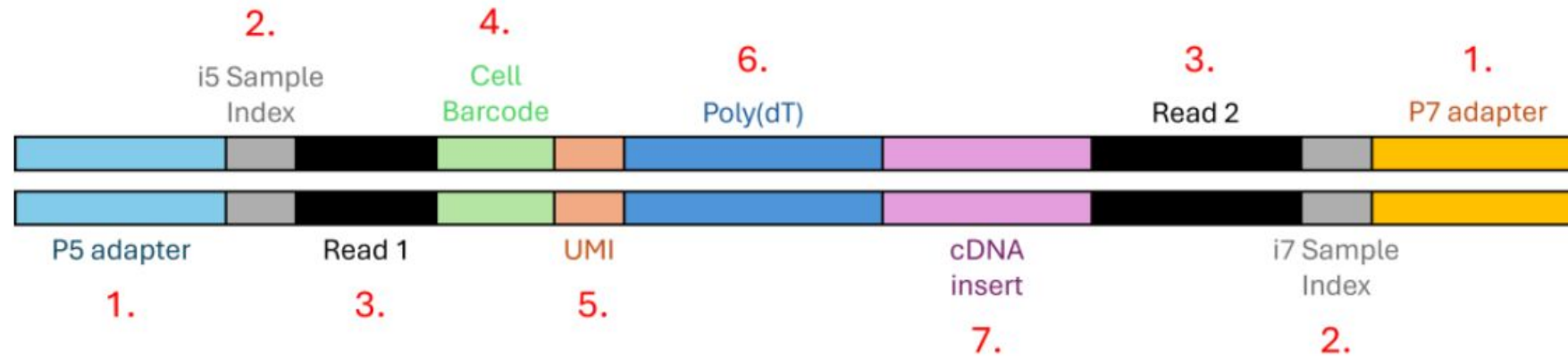
- **cellranger mkfastq** demultiplexes raw base call (BCL) files generated by Illumina sequencers into FASTQ files. It is a wrapper around Illumina's bcl2fastq, with additional features that are specific to 10x Genomics libraries and a simplified sample sheet format.

FASTQ consists of many group of 4 lines like this:

```
@SEQ ID
GATTTGGGGTTCAAAGCAGTATCGATCAAATAGTAAATCCATTTGTTCAACTCACAGTTT
+
!''*(((((***+))%%%++) (%%%)) .1***-+*'' ) **55CCF>>>>>CCCCCCC65
```

<https://support.10xgenomics.com/single-cell-gene-expression/software/pipelines/latest/what-is-cell-ranger>

[illegible]

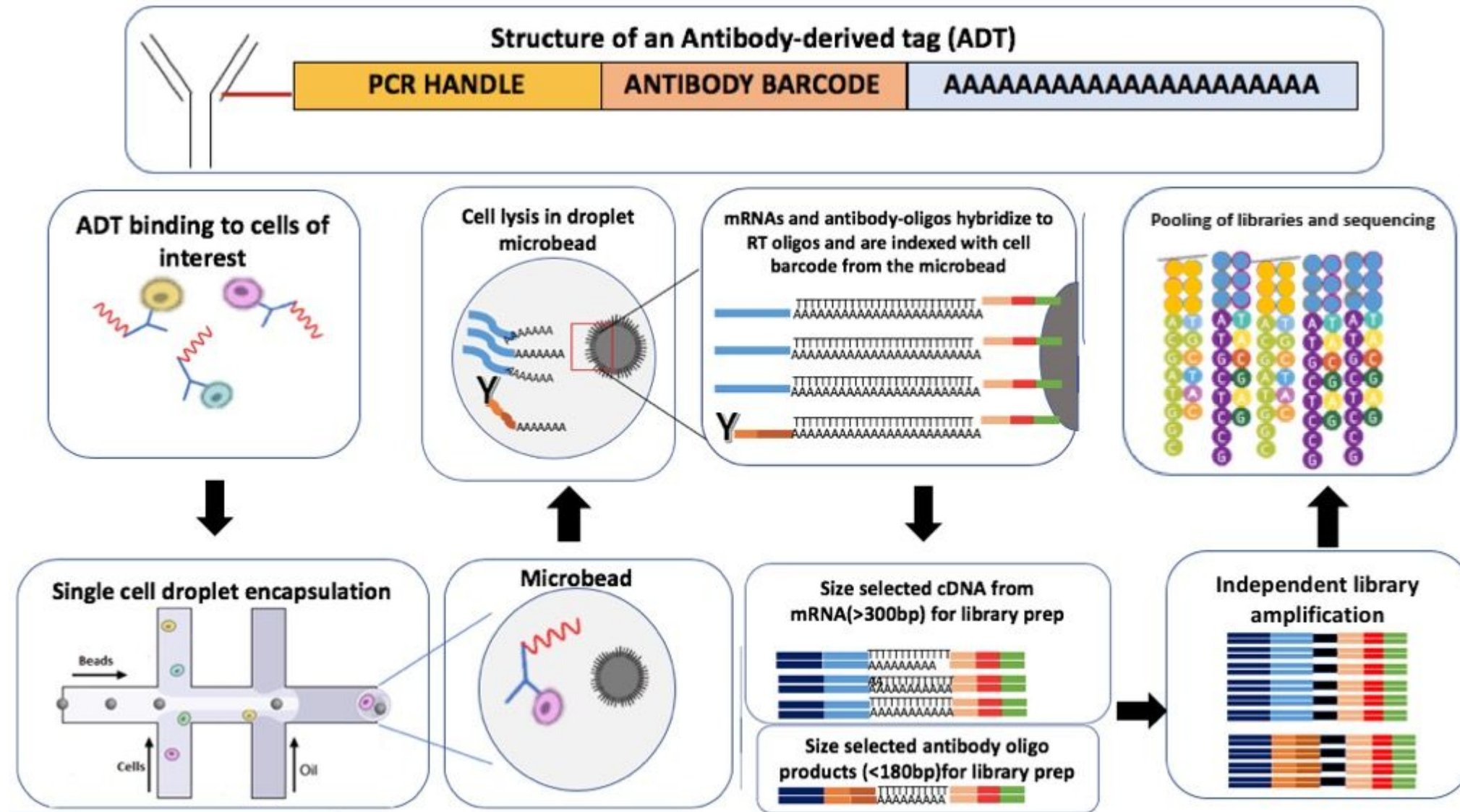


- **cellranger count** takes FASTQ files from `cellranger mkfastq` and performs alignment, filtering, barcode counting, and UMI counting. It uses the Chromium cellular barcodes to generate feature-barcode matrices, determine clusters, and perform gene expression analysis. The `count` pipeline can take input from **multiple sequencing runs** on the same **GEM well**. `cellranger count` also processes **Feature Barcode** data alongside Gene Expression reads.

<https://biotech.wisc.edu/guide-to-getting-started-with-10x-genomics-single-cell-rna-seq>

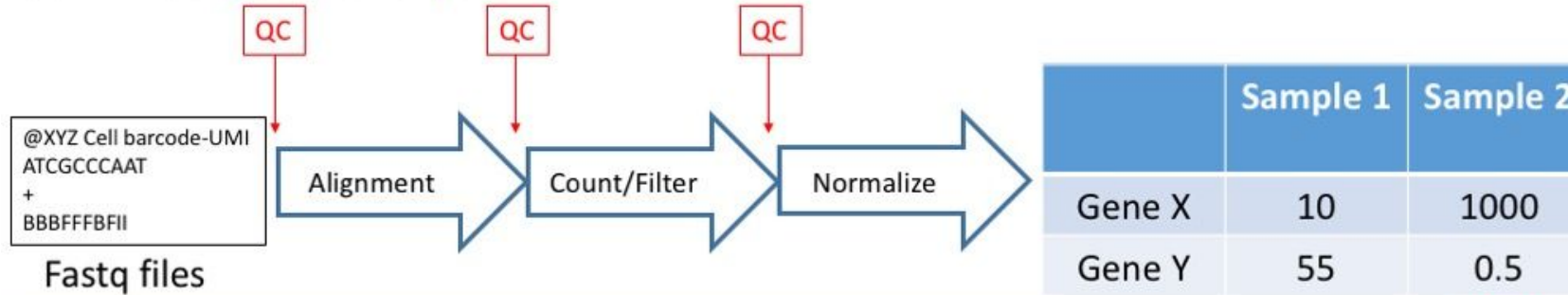
<https://support.10xgenomics.com/single-cell-gene-expression/software/pipelines/latest/what-is-cell-ranger>

CITE-seq using 10X

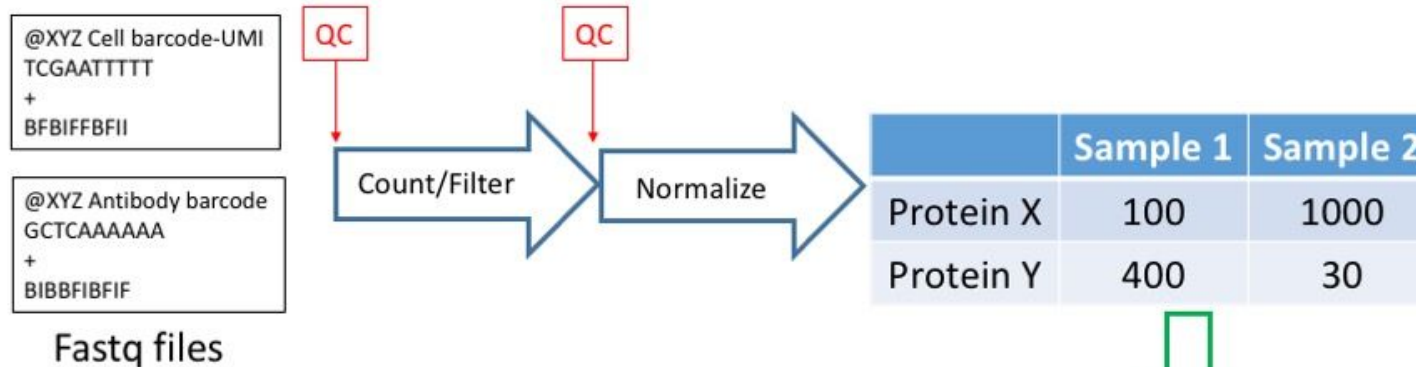


CITE-seq data analysis

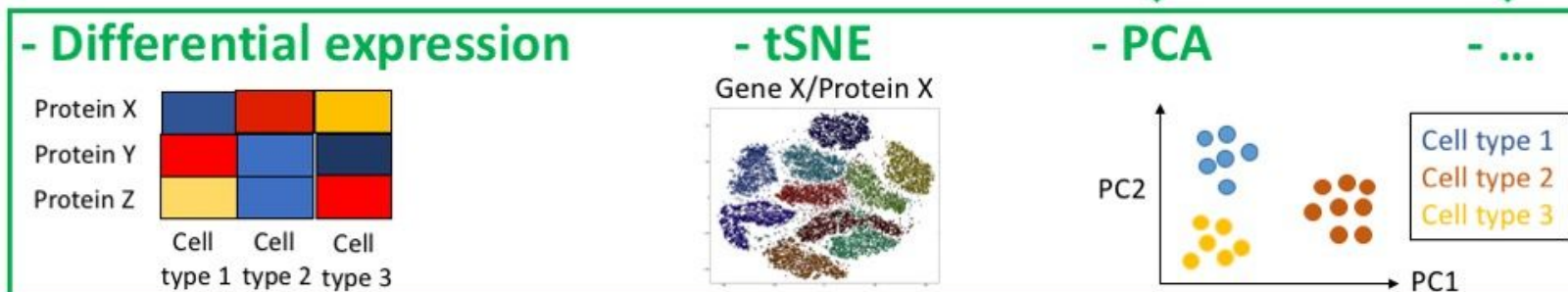
scRNA-Seq data analysis



ADT data analysis



<https://en.wikipedia.org/wiki/CITE-Seq>



Go to CITE-seq tutorial:

https://satijalab.org/seurat/articles/multimodal_vignette

Data download pointers at:

https://github.com/steverozen/cite_and_atac_wkshp/tree/main

See next slide for local paths; tutorial has absolute paths that are not on our computers

Note error in last section of tutorial

Load in the RNA UMI matrix

Note that this dataset also contains ~5% of mouse cells, which we can use as negative
controls for the protein measurements. For this reason, the gene expression matrix has
HUMAN_ or MOUSE_ appended to the beginning of each gene.

```
cbmc.rna <- as.sparse(read.csv(file = "GSE100866_CBMC_8K_13AB_10X-RNA_umi.csv.gz",  
  sep = ",", header = TRUE, row.names = 1))
```

To make life a bit easier going forward, we're going to discard all but the top 100 most
highly expressed mouse genes, and remove the 'HUMAN_' from the CITE-seq prefix

```
cbmc.rna <- CollapseSpeciesExpressionMatrix(cbmc.rna) # Seurat function
```

Load in the ADT UMI matrix ADT = "antibody derived tags"

```
cbmc.adt <- as.sparse(read.csv(file = "GSE100866_CBMC_8K_13AB_10X-ADT_umi.csv.gz",  
  sep = ",", header = TRUE, row.names = 1))
```

Note that since measurements were made in the same cells, the two matrices have identical
column names

```
all.equal(colnames(cbmc.rna), colnames(cbmc.adt))
```

The final section, "Loading data from 10X multi-modal experiments", demonstrates the more common format for CITE-seq data coming out of Cell Ranger

The earlier CBMC section reads two pre-separated CSV files (one RNA, one ADT) from a GEO submission.

That's atypical. In practice, 10x Genomics' Cell Ranger outputs a single combined feature-barcode matrix that interleaves Gene Expression and Antibody Capture features, and `Read10X()` returns it as a named list keyed by feature type.

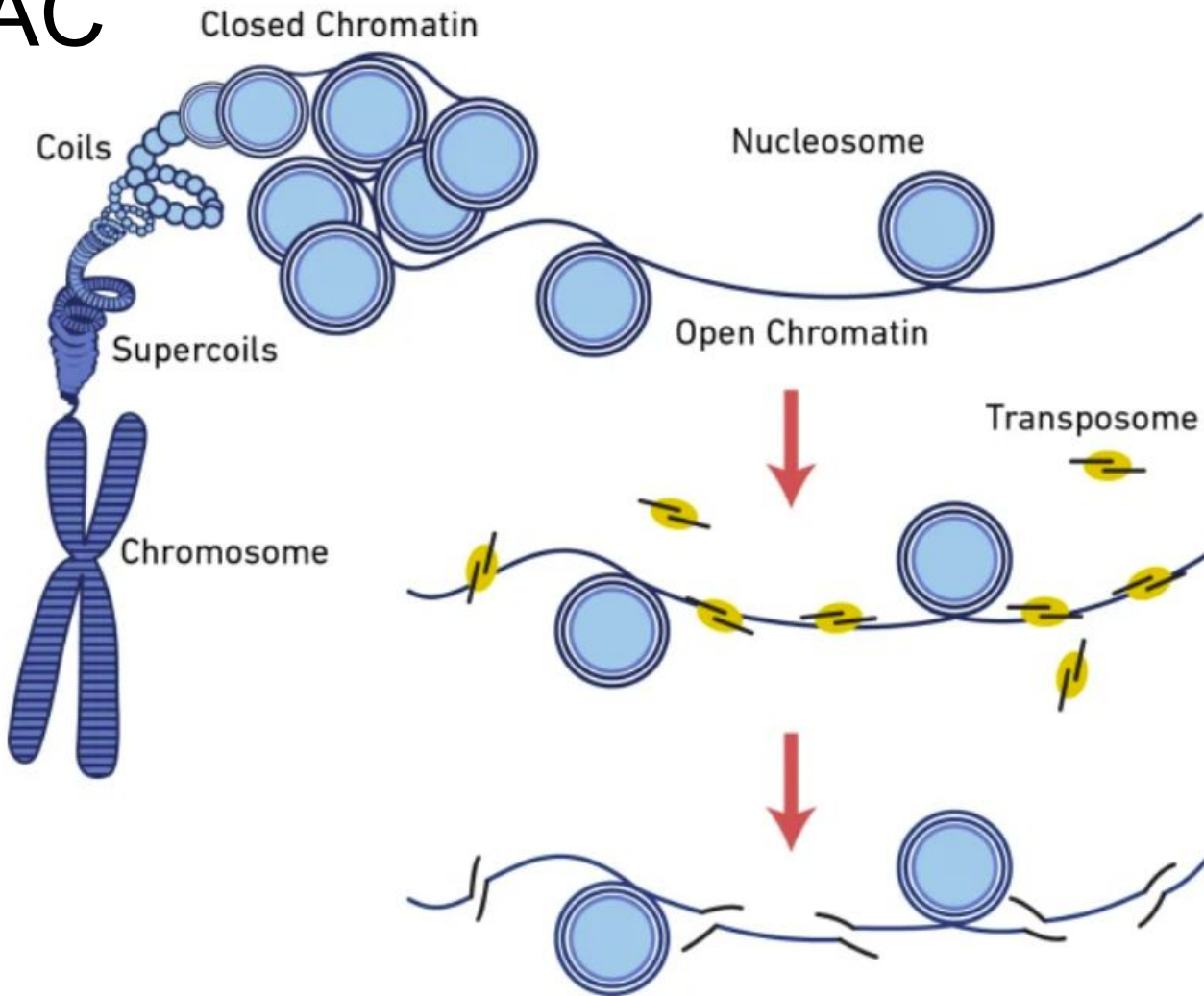
The final section shows how to

1. Untar a Cell Ranger output and read it with `Read10X()`.
2. Pull the two modalities apart from the returned list (`[["Gene Expression"]]` and `[["Antibody Capture"]]`).
3. Tidy up antibody names (stripping `_TotalSeqB` / `_control_TotalSeqB` suffixes).
4. Build a multimodal Seurat object the same way as before
5. Then do CLR-normalized ADT scatter plots (CD19/CD3, CD4/CD8a, and ADT-CD3 vs RNA-CD3E) as a sanity check that protein and RNA agree

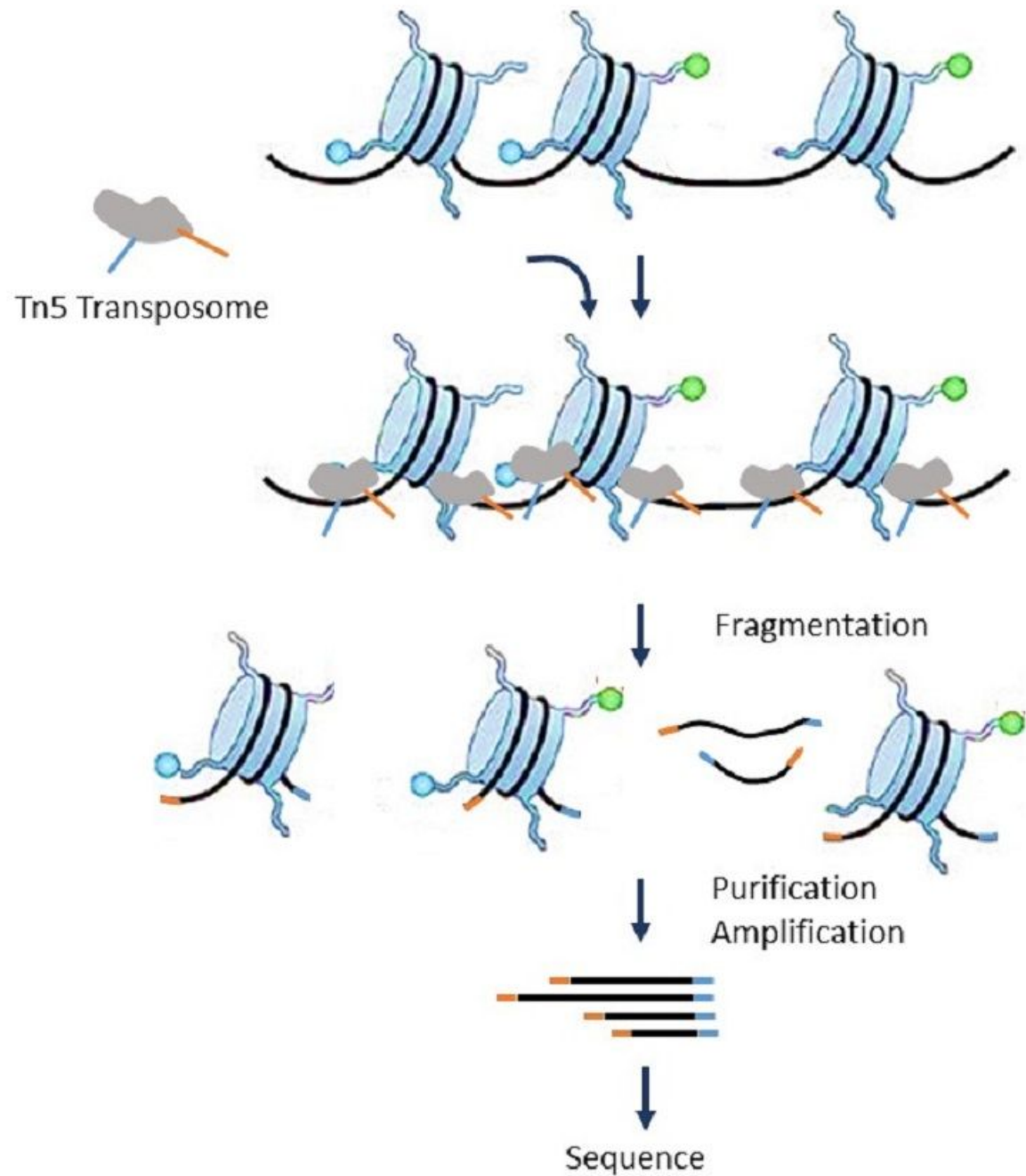
Same workflow as the CBMC example, but starting from the file format you'll get from your own 10x run.

2 Single cell ATAC-seq,
integrated with sc RNA-seq

“Assay for Transposase Accessible Chromatin” ATAC

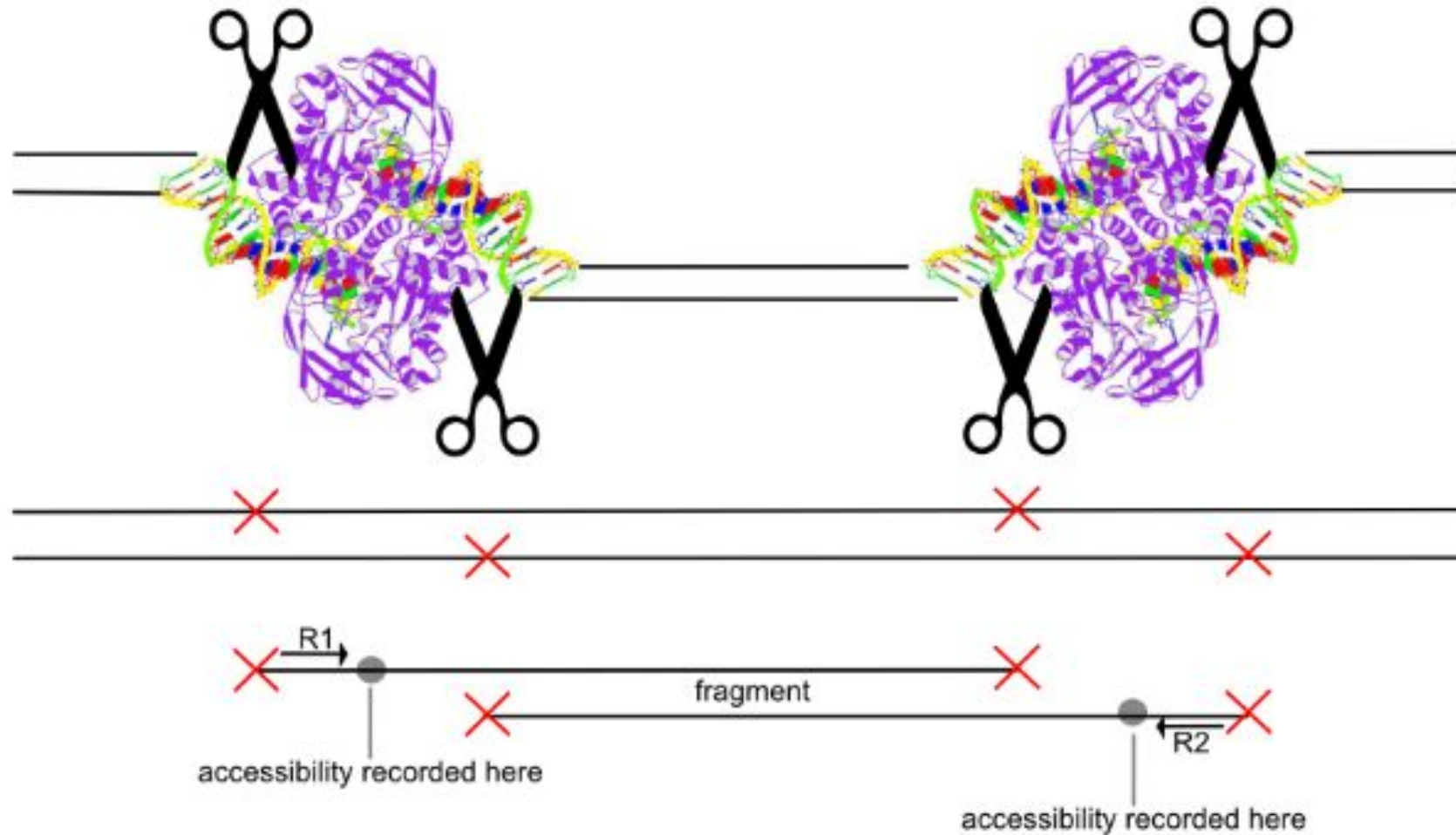


<https://www.10xgenomics.com/blog/is-single-cell-epigenomics-right-for-me-atac-ing-your-research-questions-for-deeper-insights>



<https://www.france-genomique.org/technological-expertises/regulome/cartographie-des-sites-daccessibilite-de-la-chromatine-at-ac-seq/?lang=en>

Tn5 transposase – bind and cut



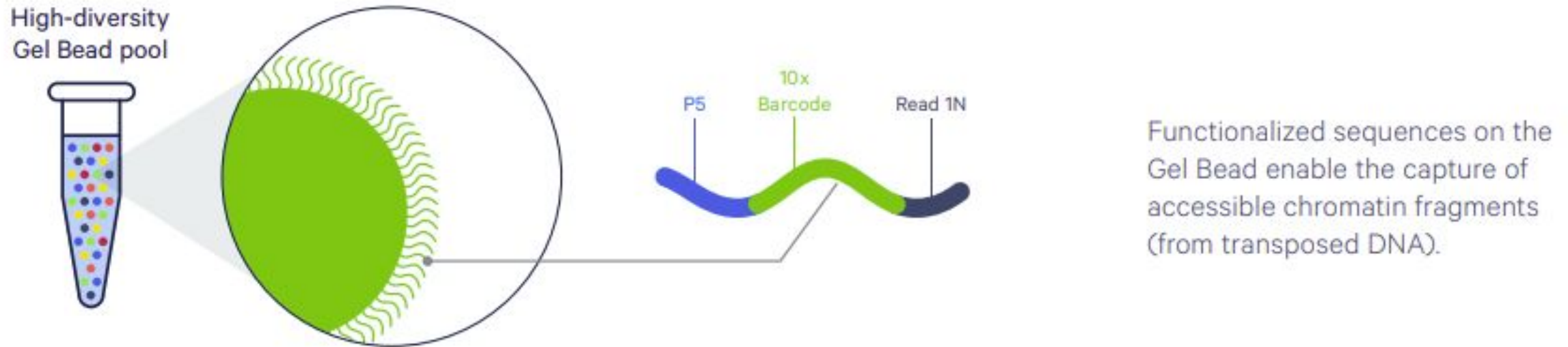
Transposase image accessed from the Protein Data Bank, <https://rcsb.org/structure/1MUH>

<https://support.10xgenomics.com/single-cell-atac/software/pipelines/latest/algorithms/overview#peaks>

Chromium Single Cell ATAC

Chromium Single Cell ATAC (Assay for Transposase Accessible Chromatin) allows you to analyze chromatin accessibility at the single cell level, providing insights into cell types and states and a deeper understanding of gene regulatory mechanisms.

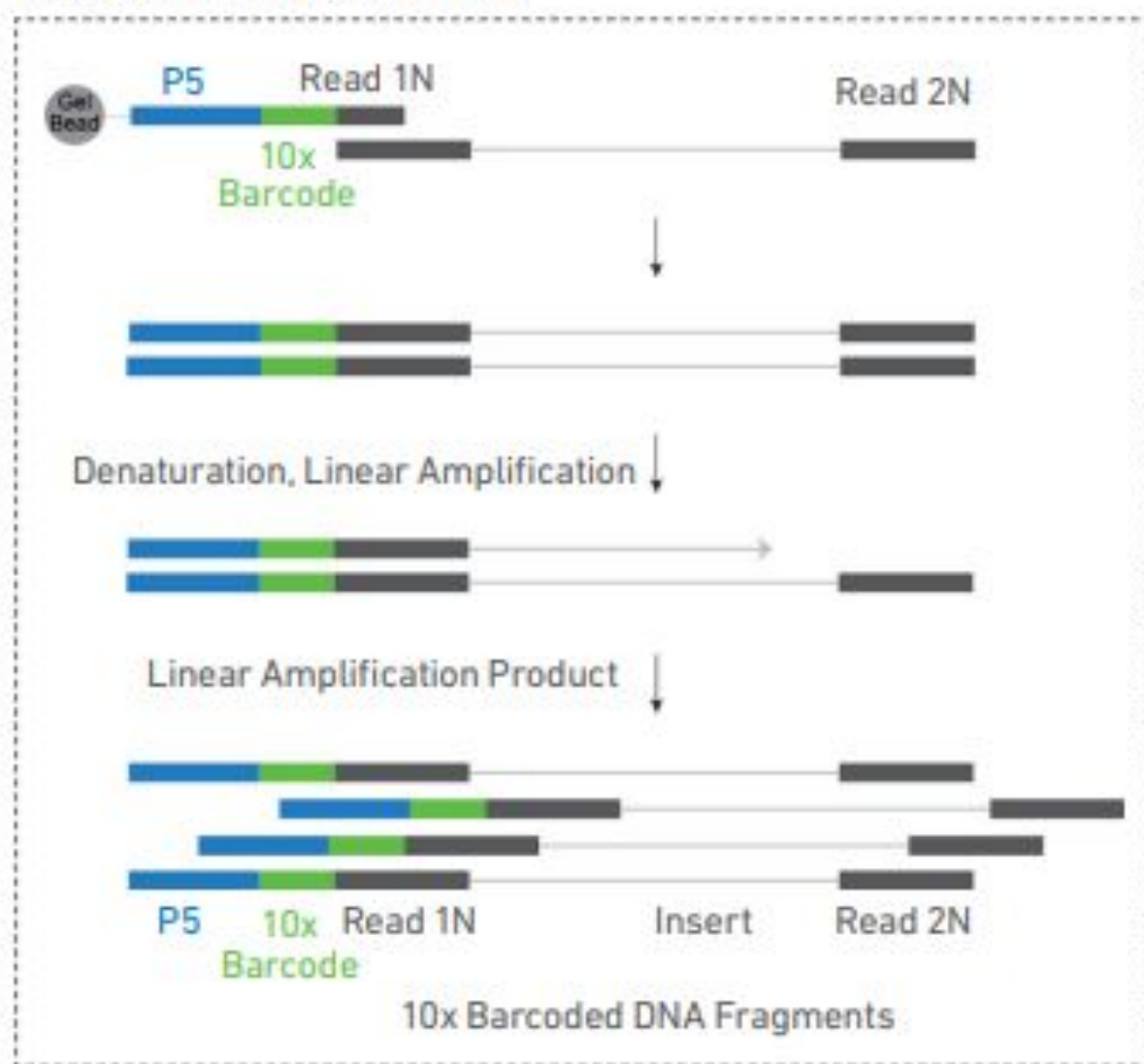
Assay input: Nuclei



https://gtac.wustl.edu/wp-content/uploads/10x_LIT025_Inside-Chromium-Next-GEM-Technology_Letter_Digital.pdf (dead link)

see also <https://www.youtube.com/watch?v=ufUVMHLDa00>

Inside Individual GEMs



We will analyze data generated by the 10X “Cell Ranger ATAC v2” Software;

This software handles

Adapter trimming

Alignment

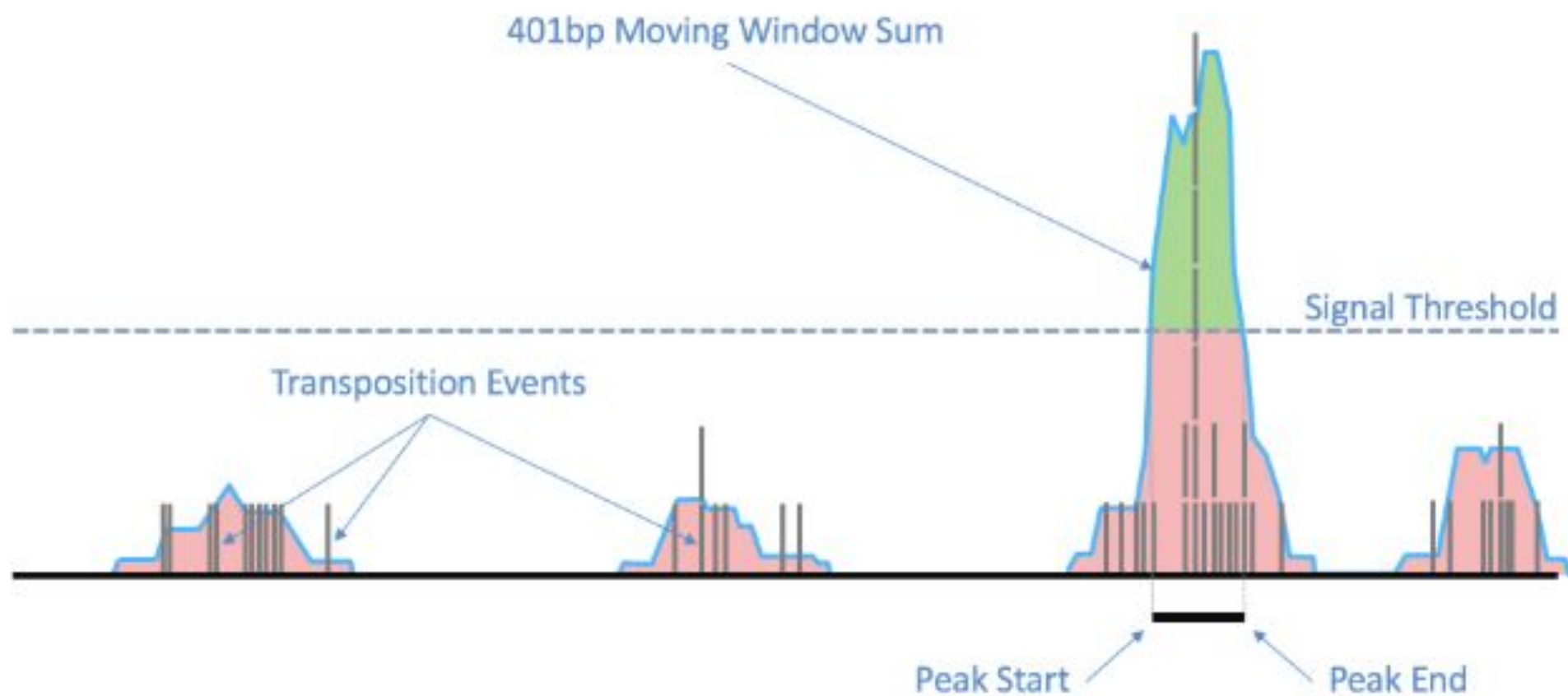
Duplicate marking

Creating “fragments file”

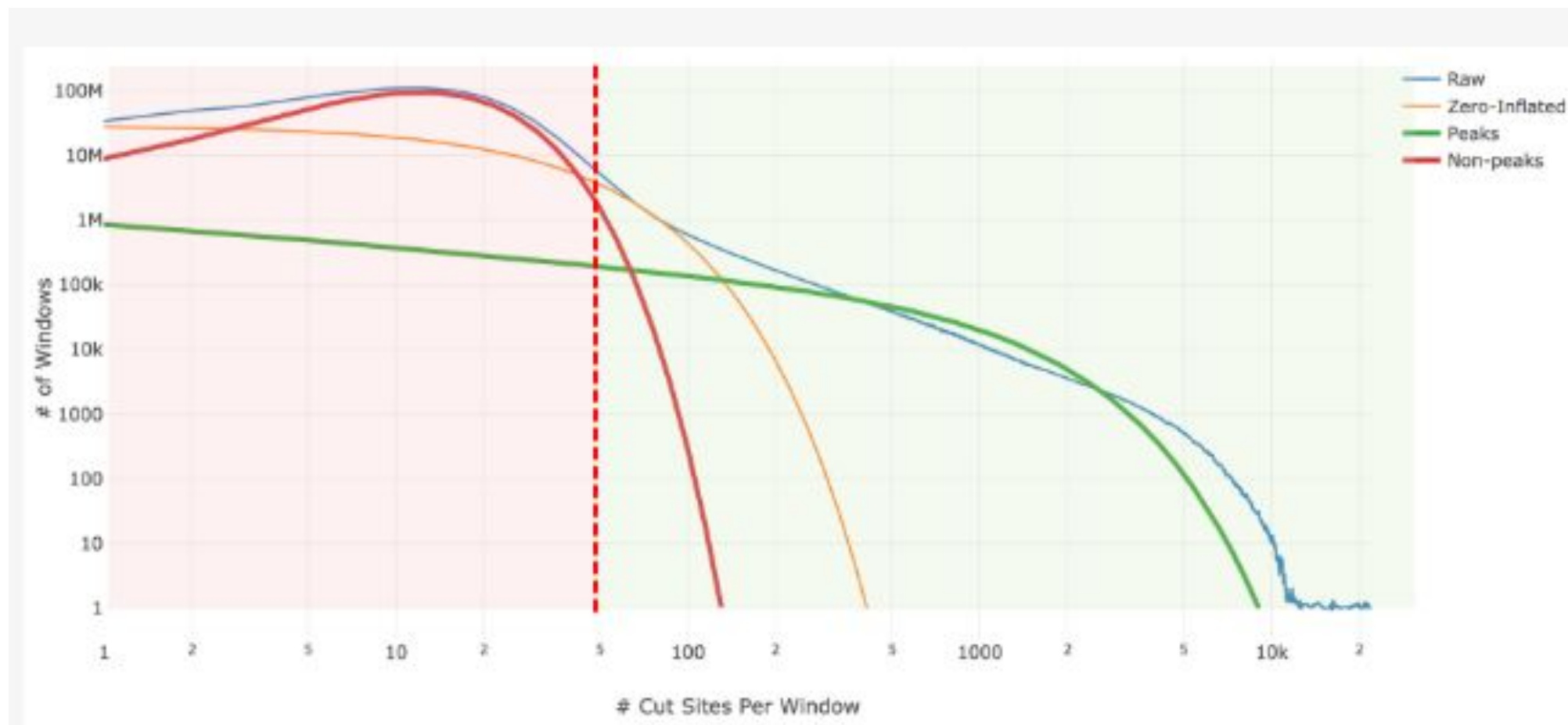
Peak calling

Algorithm overview:

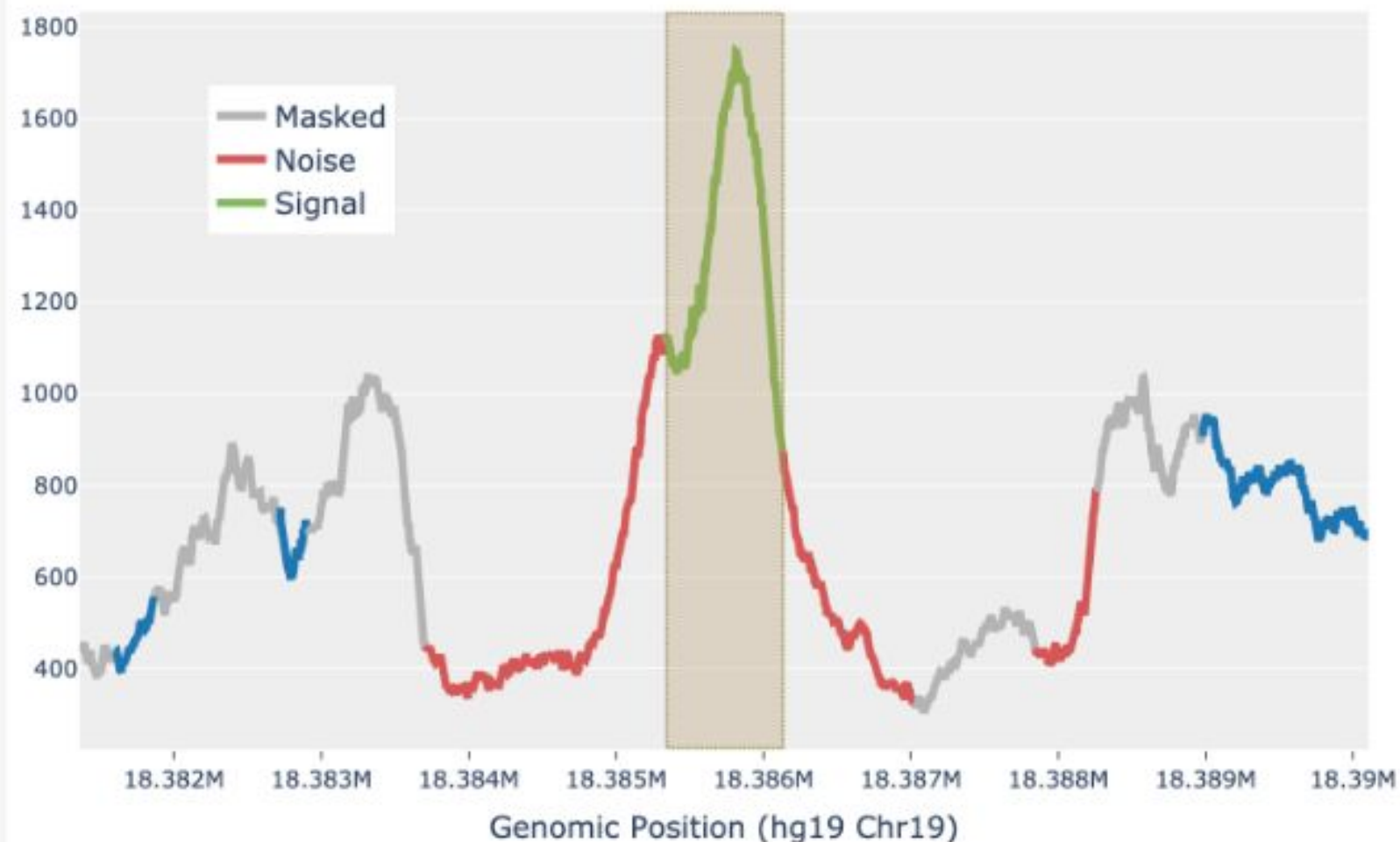
<https://support.10xgenomics.com/single-cell-atac/software/pipelines/latest/algorithms/overview>



- i** Above: Raw transposition events are used to produce a local smoothed signal track with a 401bp moving window sum. After fitting and selecting a global peak threshold, contiguous regions with signal above the threshold (shown in green) are produced as candidate peak calls.



Above: A diagram of the three-component fitting process for setting the initial global peak threshold. The x-axis shows (in logarithmic scale) the count of cut sites near a particular genomic locus, while the y-axis shows (again in logarithmic scale) the number of genomic windows with that cut-site count. The blue line is the observed data from our sample which the algorithm attempts to fit. The other three lines are the final fit: orange shows the geometric zero-inflated component, red the negative binomial non-peak background component, and green the negative binomial peak component. The sum of these three components closely approximates the empirical blue curve. The peak threshold (vertical red line) is set so at least 95% of the non-peak components are less than the threshold.



i Above: a diagram of how the local signal-to-noise estimate is performed for a single putative peak in a candidate region. The green section of the signal shows the putative peak under examination, with the peak signal measured as the median value across the green section. The grey sections are masked out, as they are other putative peaks and so are not used to estimate the local background. The red sections are used for local background estimates, with the peak background as the median value across all red sections.

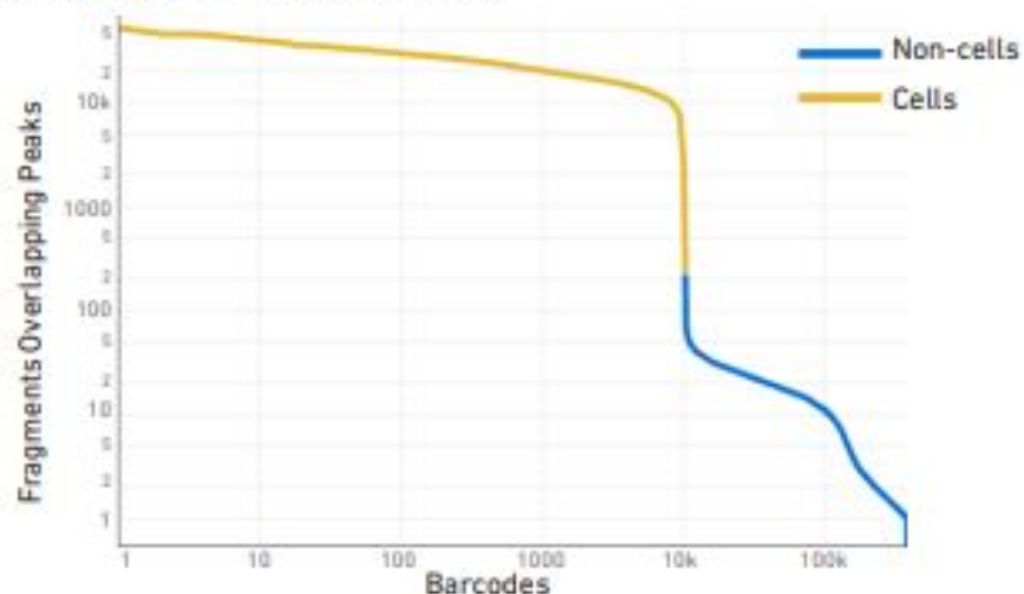
Table 2. Plots in the ATAC web summary file.

Plot & Interpretation

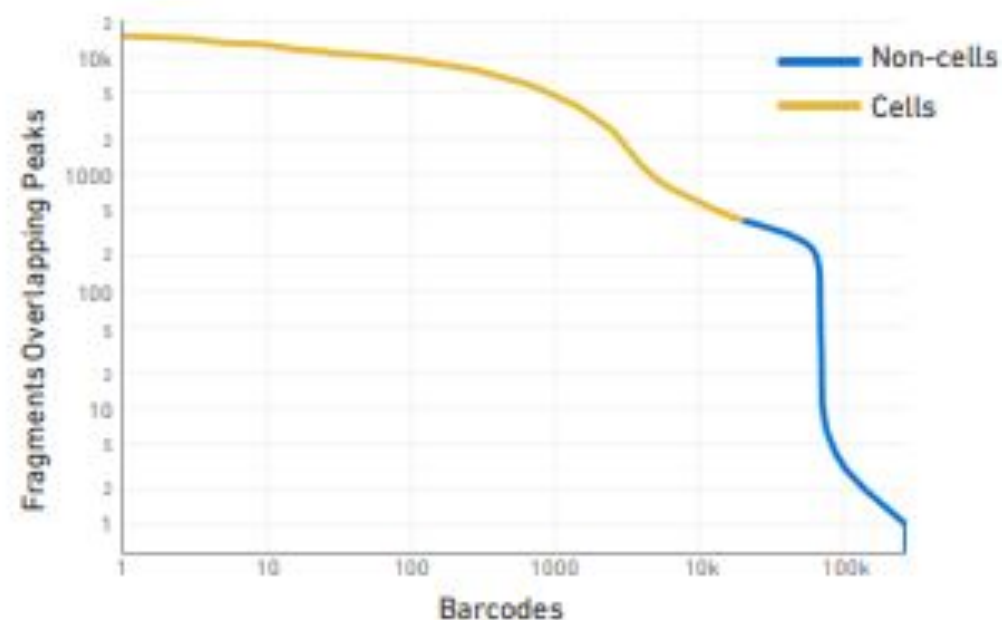
Barcode Rank Plot: The Barcode Rank (or knee plot) for fragments overlapping peaks marks the barcodes that were inferred to be associated with cells.

Example

Ideal Sample: A steep drop-off is indicative of good separation between the cell-associated barcodes and the barcodes associated with empty GEMs.



Compromised Sample: Round curve and lack of steep drop-off may indicate low sample quality or loss of single-cell behavior.



Output files from Cell Ranger ATAC v2 software that we will analyze are at

https://github.com/steverozen/cite_and_atac_wkshp/tree/main

10k_pbmc_ATACv2_nextgem_Chromium_Controller - Human PBMCs

For guidance, please consult "[Interpreting Cell Ranger ATAC Web Summary Files](#)" or contact 10x Genomics Support (support@10xgenomics.com).

10,246

Estimated number of cells

22,226

Median high-quality fragments per cell

65.0%

Fraction of high-quality fragments overlapping
peaks

Summary

Data Quality

Sample

Sample ID	10k_pbmc_ATACv2_nextgem_Chromium_Controller
Sample description	Human PBMCs
Pipeline version	cellranger-atac-2.1.0
Reference path	...ata-cellranger-arc-GRCh38-2020-A-2.0.0
Chemistry	ATAC
Organism	Homo_sapiens

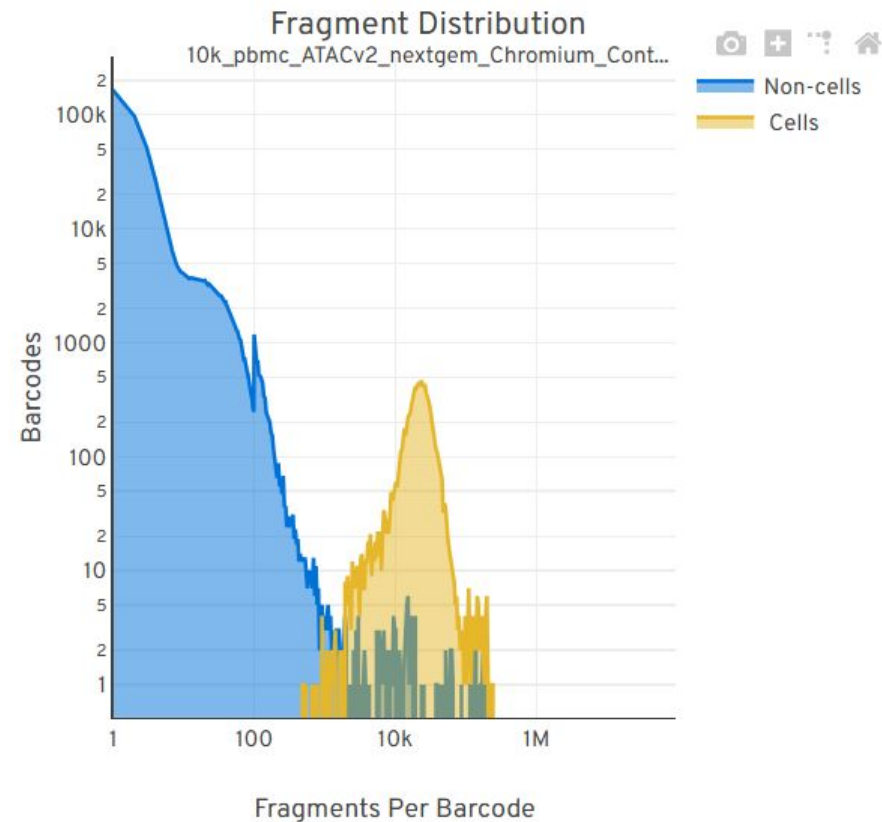
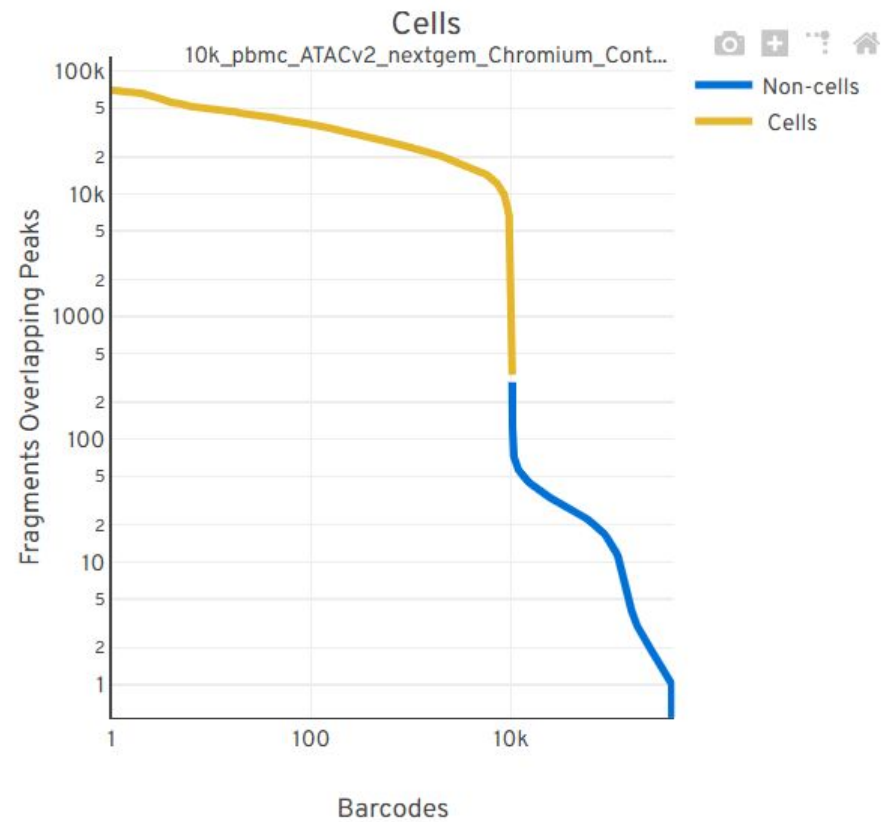
Sequencing ?

Sequenced read pairs	566,333,738
Valid barcodes	96.5%
Q30 bases in barcode	90.0%
Q30 bases in read 1	95.1%
Q30 bases in read 2	94.5%
Q30 bases in sample index i1	93.2%

https://cf.10xgenomics.com/samples/cell-atac/2.1.0/10k_pbmc_ATACv2_nextgem_Chromium_Controller/10k_pbmc_ATACv2_nextgem_Chromium_Controller_web_summary.html

Cells ?

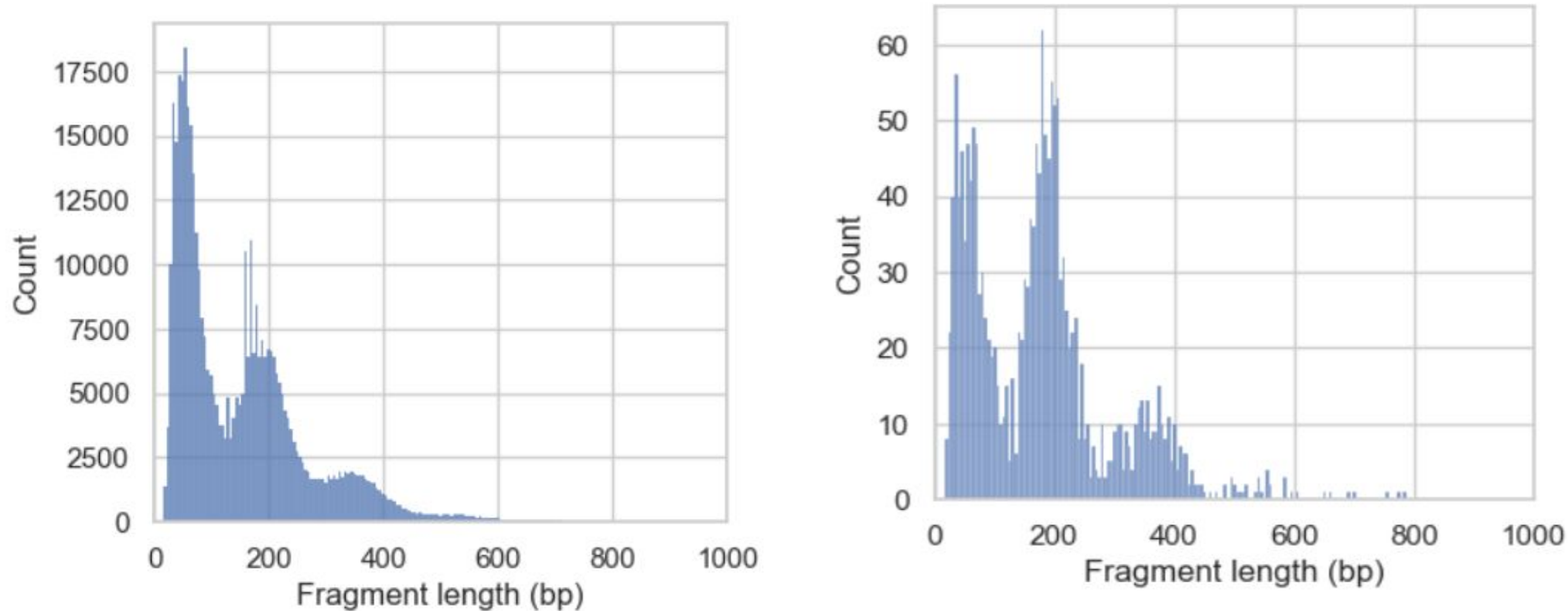
Estimated number of cells	10,246
Mean raw read pairs per cell	55,273.64
Fraction of high-quality fragments in cells	95.5%
Fraction of transposition events in peaks in cells	61.6%
Median high-quality fragments per cell	22,226



Tutorial at:

https://stuartlab.org/signac/articles/pbmc_vignette.html

Notes for the tutorial: ATAC seq QC



In the first histograms, we see periodically decreasing peaks corresponding to fragment lengths in nucleosome-free regions (<100bp), mono-, di- and a small tri-nucleosomal length. In high quality ATAC-seq data we expect an enrichment of fragments from nucleosome-free regions compared to fragments with mono- and multi-nucleosomal lengths (this ratio is calculated as the nucleosome signal). In the second plot this ratio sub-optimal, and we exclude these cells. However, the low number of cells with a high nucleosome signal indicate that we are working with data with overall good quality.

Notes for tutorial: ATAC seq analysis uses singular value decomposition, not principal components analysis

Principal Component Analysis (**PCA**) and Singular Value Decomposition (**SVD**) are closely related methods for dimensionality reduction, but they differ in their mathematical scope and how they handle data. 

The fundamental relationship is that **SVD is a mathematical algorithm** used to factorize any matrix, whereas **PCA is a statistical procedure** that often uses SVD as a step in its calculation. 

<https://share.google/aimode/fhtCZgludrfw40qhQ>

Notes for tutorial: ATAC seq analysis uses singular value decomposition, not principal components analysis

Key Differences

Feature 	Principal Component Analysis (PCA)	Singular Value Decomposition (SVD)
Core Goal	Summarizes data by finding directions of maximum variance.	General-purpose matrix factorization into three specific component matrices.
Data Centering	Requires data to be mean-centered (subtracting the average from each feature).	Does not require centering; it can be applied to any matrix directly.
Input Matrix	Typically applied to the covariance matrix of the data.	Applied directly to the raw data matrix .
Sparsity	Centering makes sparse data dense, making PCA computationally expensive for sparse sets.	Can be applied directly to sparse matrices without losing their computational benefits.

<https://share.google/aimode/fhtCZgludrfw40qhQ>

Notes for tutorial: ATAC seq analysis uses singular value decomposition, not principal components analysis

The Mathematical Link

Mathematically, **PCA is essentially SVD performed on mean-centered data.** 

- If you center your data (subtract the mean) and then apply SVD, the resulting right-singular vectors (matrix V) are the **Principal Components**, and the squared singular values (from matrix Σ) are proportional to the **eigenvalues** (variance) of the covariance matrix.
- Modern software (like **scikit-learn's PCA**) usually implements PCA using SVD because it is numerically more stable and efficient than explicitly calculating a covariance matrix. 

When to Use Which?

- **Use PCA** when you want a statistical interpretation of variance and your data is dense.
- **Use SVD** (specifically **Truncated SVD**) for very large or **sparse datasets** (like text data or recommendation systems) where centering would create an unmanageably large dense matrix. 

<https://share.google/aimode/fhtCZgludrfw40qhQ>

Term Frequency-Inverse Document Frequency (TF-IDF)

TF-IDF is a statistical weighting scheme that measures how important a word is to a specific document within a larger collection (corpus). It balances two metrics: 

- **Term Frequency (TF):** Measures how often a word appears in a document. High frequency suggests the word is a key topic of that page.
- **Inverse Document Frequency (IDF):** Measures how rare a word is across the entire corpus. Common words like "the" or "is" receive an IDF score near zero, while unique technical terms receive higher scores. 

How it works: By multiplying TF and IDF, the system highlights terms that are frequent in one document but uncommon elsewhere, effectively filtering out "noise" and identifying descriptive keywords. 

Latent Semantic Indexing (LSI)

LSI, also known as Latent Semantic Analysis (LSA), is a mathematical method that goes beyond individual words to identify **hidden (latent)** patterns and relationships between terms. 

- **Semantic Analysis:** LSI identifies words that frequently appear together in the same context, treating them as semantically related. For example, a search engine using LSI understands that a page mentioning "car," "engine," and "tires" is about "automobiles," even if that exact word isn't present.
- **The Process:** It often uses a mathematical technique called **Singular Value Decomposition (SVD)** to reduce the high-dimensional data (from TF-IDF) into a smaller, more manageable space that captures the "essence" or themes of the text. 

TF-IDF matrix input to LSI

If you have 3 documents and a total vocabulary of 5 words, your matrix would resemble this:

	word_1 (e.g., "cat")	word_2 (e.g., "the")	word_3 (e.g., "data")	...
Doc 1	0.45	0.02	0.00	...
Doc 2	0.00	0.01	0.82	...
Doc 3	0.12	0.02	0.35	...

Key Characteristics of the Matrix

- **Sparsity:** In most real-world datasets, this matrix is **highly sparse** (filled with zeros) because a single document only uses a tiny fraction of the total words in the entire collection.
- **Importance Weighting:**
 - **High Scores:** Words that appear frequently in one document but rarely in others (like "data" in a technical doc) get a **high value**.
 - **Low Scores:** Common words like "the" or "is" appear in almost every document, so their **IDF** pulls their score toward zero.
- **Fixed Dimensions:** Every document vector in the matrix will have the same length (equal to the number of unique words in your entire set), making them easy to compare mathematically. 

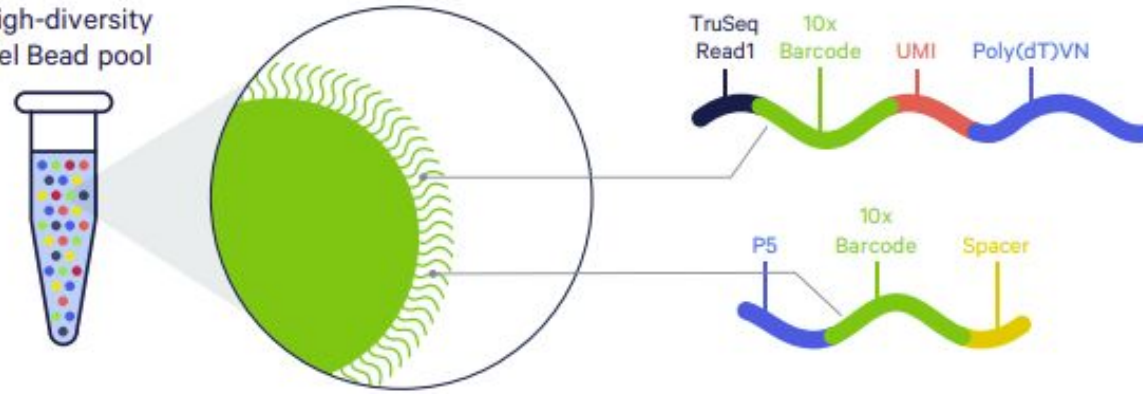
Backup Slides

Chromium Single Cell Multiome ATAC + Gene Expression

Simultaneously profile gene expression and open chromatin from the same cell with Chromium Single Cell Multiome ATAC + Gene Expression. Multiply your power of discovery to characterize cell types and states, and uncover gene regulatory programs.

Assay input: Nuclei

High-diversity
Gel Bead pool



Functionalized sequences on the Gel Bead enable the dual capture of accessible chromatin fragments (from transposed DNA) and 3' mRNA.

Some good references:

<https://bioconductor.org/books/release/OSCA/>

<https://www.sc-best-practices.org/preamble.html>

<https://www.nature.com/articles/s41587-025-02694-w>

(<https://openproblems.bio/>)

What is a K-Nearest Neighbor (KNN) Graph?



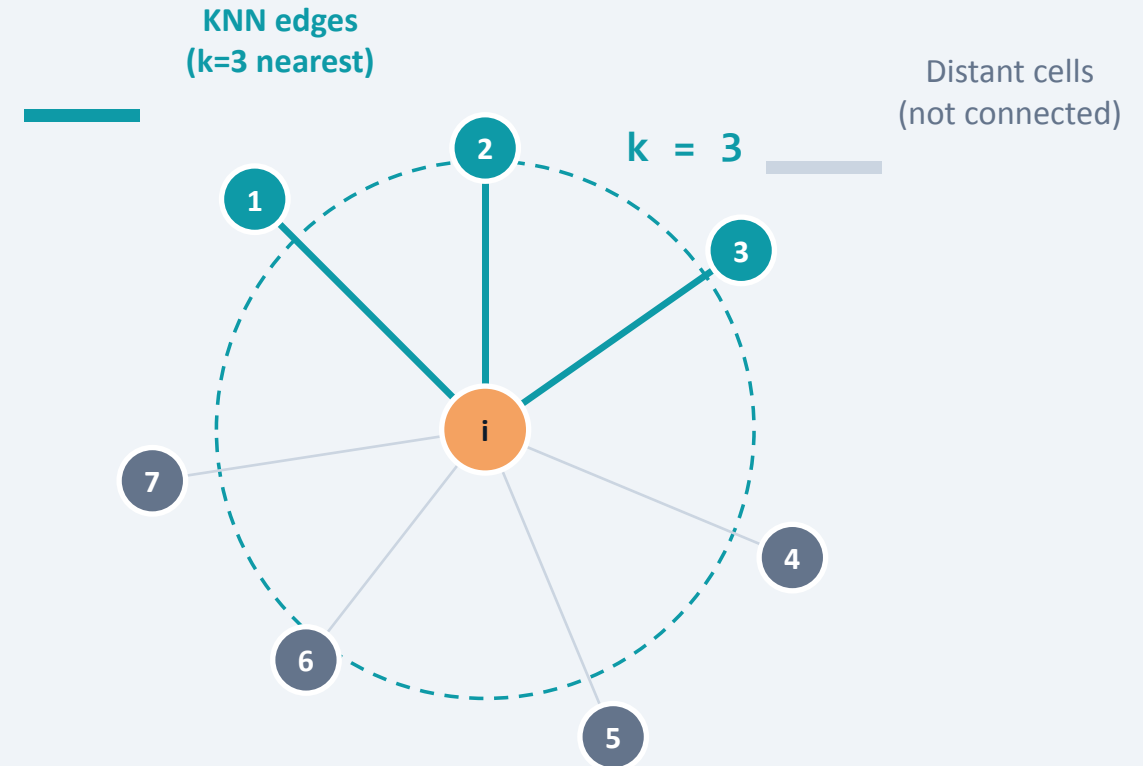
Definition

Given a set of n cells, each represented as a point in PCA space:

For every cell i , find its k most similar cells (by Euclidean distance).

Draw a directed edge from i to each of its k neighbors.

The result is the KNN graph: a sparse representation of local structure in high-dimensional expression space.



KNN Graph: Key Properties & Limitations



Distance metric

Seurat uses Euclidean distance in PCA space. The top N PCs capture most biologically meaningful variance, so distances there are more reliable than in raw gene expression space.



Directed vs. undirected

KNN is initially directed ($i \rightarrow j$ does not imply $j \rightarrow i$). Seurat symmetrizes it: an edge exists if either cell nominated the other as a neighbor.



Choosing k

Default is $k=20$. Larger $k \rightarrow$ smoother, more connected graph (favors larger clusters). Smaller $k \rightarrow$ sparser graph (can resolve rare cell types but more noise-sensitive).



Core limitation

Edge weights are binary (0 or 1). Two cells that share 1 neighbor out of 20 look the same as two cells sharing 19 of 20 neighbors. This discards valuable similarity information.



Uneven density problem

In heterogeneous tissues, dense populations (e.g. abundant T cells) and sparse ones (e.g. rare progenitors) have very different neighborhood structures. Binary KNN treats them equally.

From KNN to Shared Nearest Neighbor (SNN): Refining Edge Weights



The core idea: two cells are more similar if they share many of the same neighbors — not just if they are each other's neighbors.

1

Build KNN graph

Connect each cell to its k nearest neighbors in PCA space.

2

Count shared neighbors

For every pair of connected cells (i, j) , count how many neighbors they have in common: $|N(i) \cap N(j)|$.

3

Compute Jaccard similarity

Divide shared neighbors by the union of their neighbor sets:
$$\text{Jaccard}(i, j) = \frac{|N(i) \cap N(j)|}{|N(i) \cup N(j)|}$$

4

Prune weak edges

Drop edges where Jaccard $<$ threshold (default 1/15 in Seurat). This removes spurious connections.

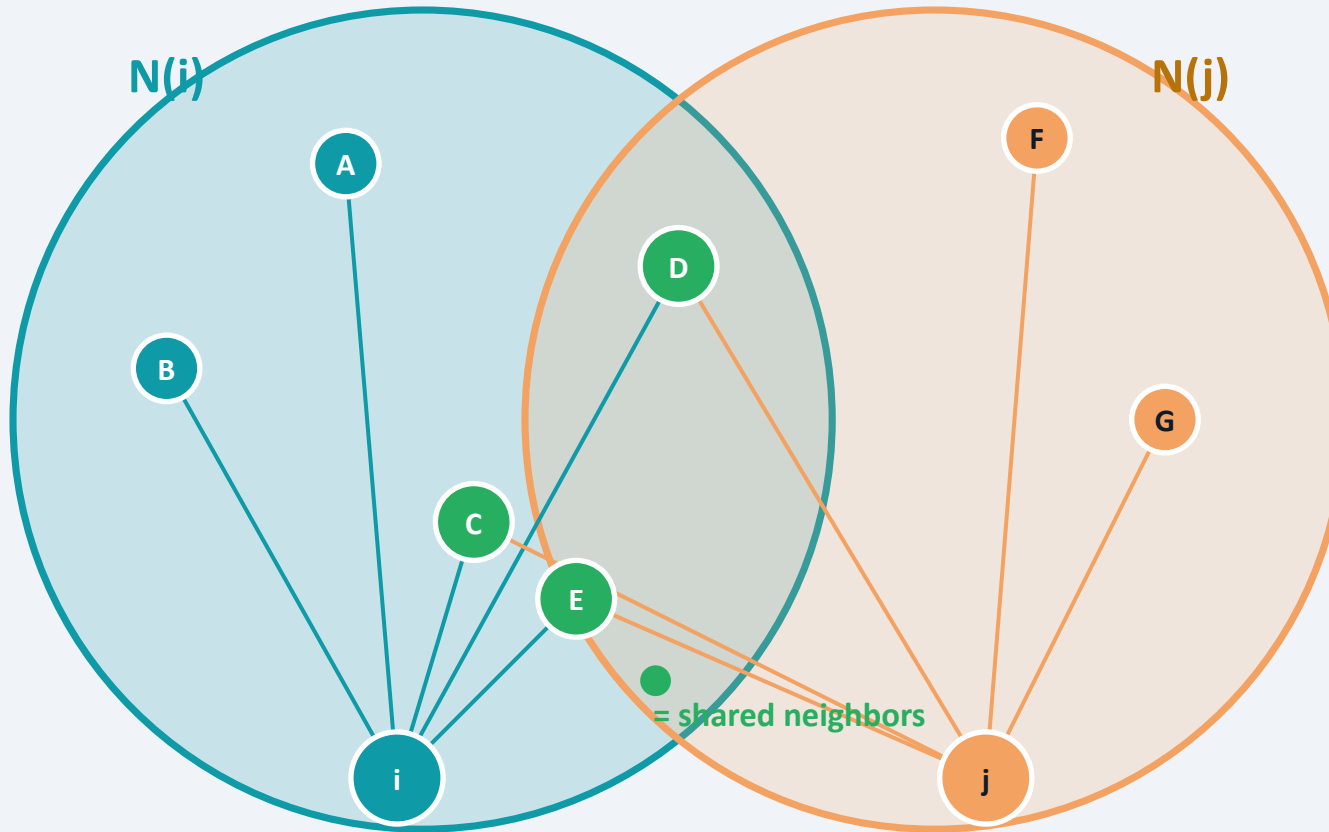
5

SNN graph ready

Remaining edges carry Jaccard weights (0–1). This is the input to Louvain/Leiden.

$$\text{Jaccard}(i, j) = \frac{|N(i) \cap N(j)|}{|N(i) \cup N(j)|}$$

Jaccard Similarity: A Worked Example



Calculation

$N(i) = \{A, B, C, D, E\}$

$N(j) = \{C, D, E, F, G\}$

Intersection:

$N(i) \cap N(j) = \{C, D, E\}$

→ 3 shared neighbors

Union:

$N(i) \cup N(j) = \{A, B, C, D, E, F, G\}$

→ 7 total neighbors

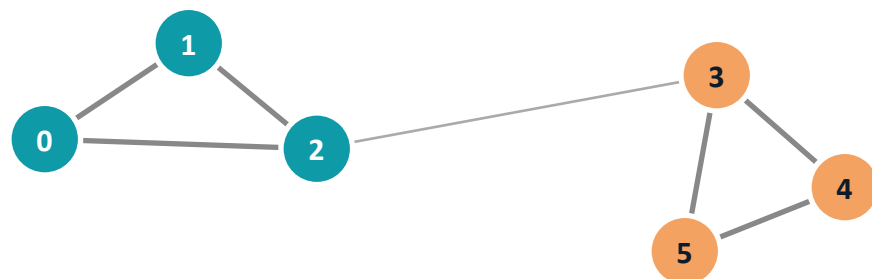
$Jaccard(i, j) = 3/7 \approx 0.43$

KNN vs. SNN: Why the Upgrade Matters



KNN Graph

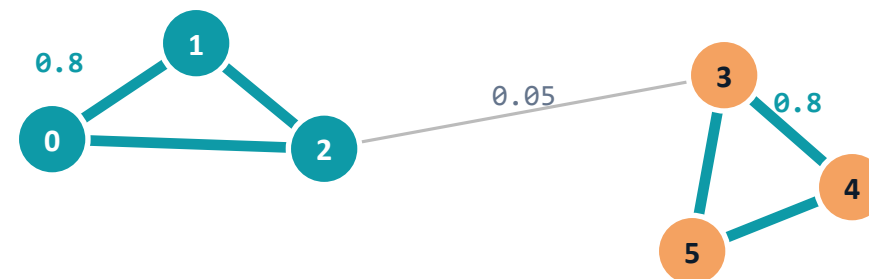
Binary edges — topology only



Bridge edge (2→3) looks identical
to strong intra-cluster edges

SNN Graph (Jaccard-weighted)

Weighted edges — similarity encoded



Bridge edge pruned (Jaccard=0.05 < threshold)
Strong intra-cluster edges clearly visible

Why SNN Works Well for scRNA-seq



01 Curse of dimensionality

scRNA-seq cells live in thousands of gene dimensions. In high dimensions, raw distances become unreliable — most cells end up roughly equidistant. Shared-neighbor counts are more robust because they rely on the rank ordering of distances, not their absolute values.

02 Dropout noise

A gene being zero in one cell but not another inflates Euclidean distance. If both cells genuinely belong to the same type, they'll still share most of the same neighbors. SNN smooths over this technical noise.

03 Uneven cluster densities

Real tissues have abundant cell types (T cells) and rare ones (progenitors). SNN naturally handles this: within a dense cluster, Jaccard scores are high; at boundaries, they drop sharply. This makes community boundaries cleaner.

04 Pruning removes spurious edges

The `prune.SNN` threshold (default 1/15) removes edges with near-zero Jaccard. This makes the graph sparser and faster to cluster, and reduces the chance of merging biologically distinct populations.

What is Louvain Clustering?



Definition

Louvain is a greedy, hierarchical algorithm for detecting communities (clusters) in networks.

Developed in 2008 by Blondel et al. at the University of Louvain (Belgium).

It maximizes a quality function called **Modularity** — measuring how densely nodes are connected within communities vs. what would be expected randomly.

Speed

Handles millions of nodes in seconds



Scalable

Sub-quadratic time complexity



Hierarchical

Produces a dendrogram of communities



No k needed

Number of clusters is auto-discovered

Modularity of Weighted Graph: The Quality Function

$$Q = (1/2m) \sum_i [w_i - s_i^2 / 2m] \cdot \delta(c_i, c_j)$$

w_{ij} Edge weight between nodes i and j (0 if no edge)

$\delta(c_i, c_j)$ 1 if nodes i and j are in the same community, else 0

s_i, s_j Strength (weighted degree) of nodes i and j : $s_i = \sum_j w_{ij}$

Q Modularity score (ranges from -0.5 to 1; higher = better communities)

m Total edge weight in graph: $m = \frac{1}{2} \sum_i s_i$

Reduces to the unweighted form when all $w_{ij} \in \{0,1\}$: then $s_i = k_i$ and $m = |E|$.

Modularity of Unweighted Graph: Quality Function



$$Q = \left(\frac{1}{2m} \right) \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \cdot \delta(c_i, c_j)$$

A_{ij}

Adjacency matrix: 1 if edge exists, 0 otherwise

$\delta(c_i, c_j)$

1 if nodes i and j are in the same community, else 0

k_i, k_j

Degree of nodes i and j

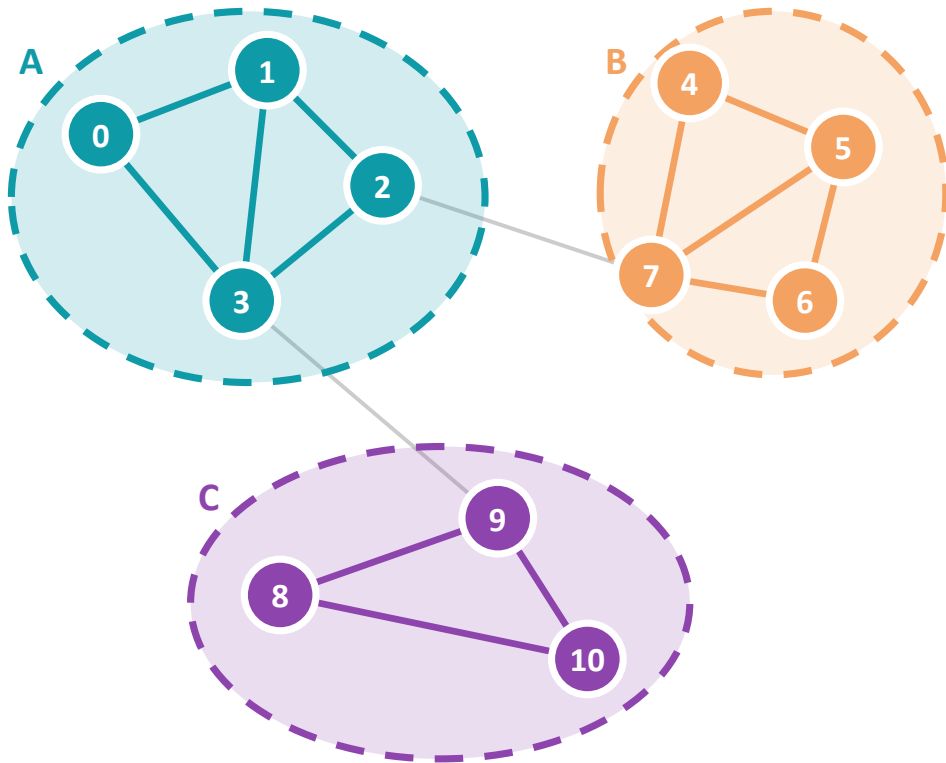
Q

Modularity score (ranges from -0.5 to 1; higher = better communities)

m

Total number of edges in the graph

Modularity in Practice: High Q



$$Q \approx 0.67$$

HIGH modularity

✓ Dense intra-community edges

Each community has many internal connections (5 edges within A, 5 within B, 3 within C).

✓ Sparse inter-community edges

Only 2 edges cross between communities — far fewer than expected by chance.

✓ $A_{ii} - k_i k_i / 2m > 0$

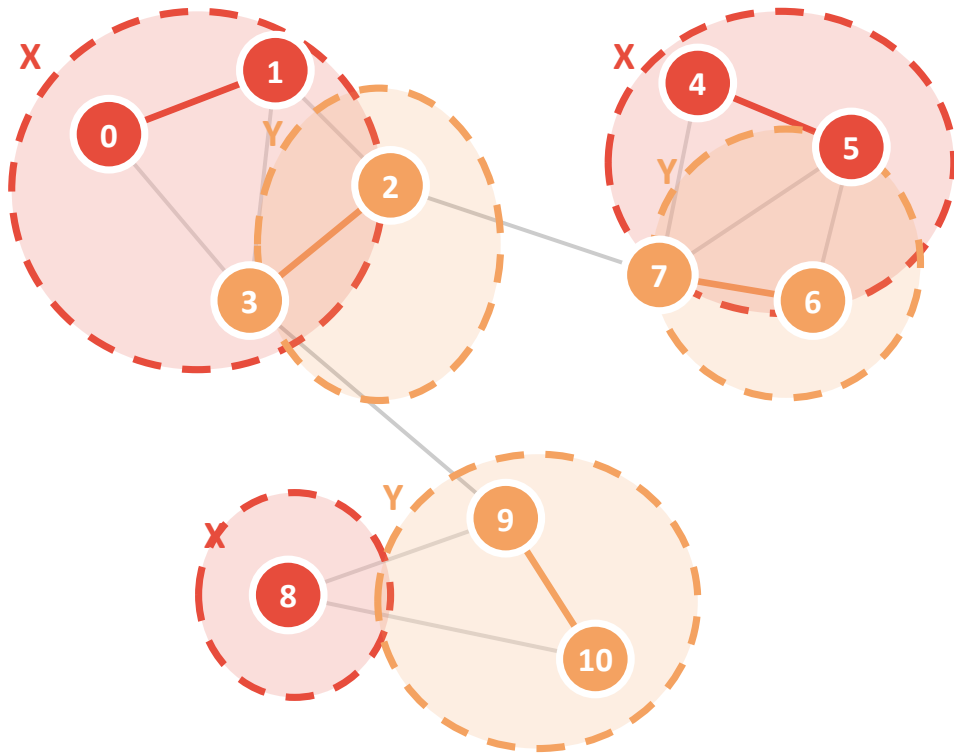
For same-community pairs, observed edges exceed the random expectation. Each term contributes positively to Q.

✓ Clear cluster separation

Community boundaries are obvious: removing the 2 bridge edges would fully disconnect the graph.

Intuition: most edges are inside communities; very few cross between them. Q rewards this structure.

Modularity in Practice: Low Q



$$Q \approx -0.09$$

LOW modularity

❌ Communities split real clusters

Each of the 3 natural clusters (A, B, C) is sliced in half — nodes 0,1 are in X while 2,3 are in Y.

❌ Many edges cross boundaries

Most of the original intra-cluster edges now run between X and Y, not within them.

❌ $A_i - k_i^2 / 2m < 0$ often

Observed edges are fewer than the random expectation for same-community pairs. Each term subtracts from Q.

❌ No benefit over random

A Q near 0 (or negative) means the partition is no better — or worse — than assigning communities at random.

Intuition: edges mostly cross community boundaries. The partition "wastes" connections — Q penalises this.

The Louvain Algorithm: Two Phases



PHASE 1 — Modularity Optimization

1. Assign each node to its own community.
2. For each node i , try moving it to each neighboring community in turn (neighbors visited in random order).
3. Compute the ΔQ gain for each candidate move.
4. Apply the move with the highest $\Delta Q > 0$; skip if none improve.
5. Repeat until no single move improves Q .

PHASE 2 — Network Aggregation

1. Compress all nodes in a community into a single 'super-node'.
2. Edge weights between super-nodes = sum of original edges.
3. Self-loops represent internal edges of the community.
4. Apply Phase 1 again on this compressed graph.
5. Repeat both phases until no further improvement.

Computing the Modularity Gain (ΔQ)



When moving node i into community C :

$$\Delta Q = \left[\sum_{i \in C} w_{ij} + 2k_{i,C} \right] / 2m - \left[\left(\sum_{i \in C} d_i + k_i \right) / 2m \right]^2 - \sum_{i \in C} w_{ij} / 2m + \left(\sum_{i \in C} d_i / 2m \right)^2 + \left(k_i / 2m \right)^2$$

$\sum_{i \in C} w_{ij}$

Sum of weights of edges inside community C

$k_{i,C}$

Sum of weights of edges from i to nodes in C

$\sum_{i \in C} d_i$

Sum of degrees of all nodes in C

m

Total weight of all edges in graph

k_i

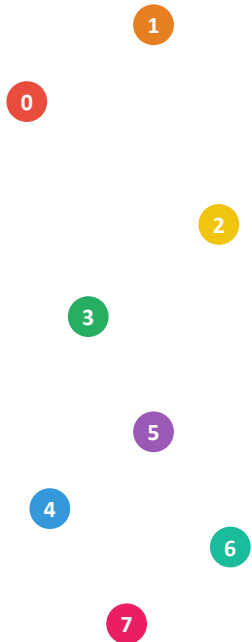
Degree (sum of edge weights) of node i

Only move node i if $\Delta Q > 0$

Louvain in Action: Visual Walkthrough

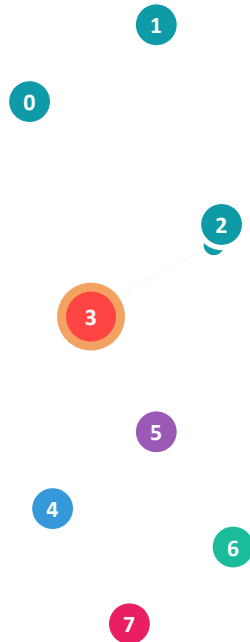


Step 1



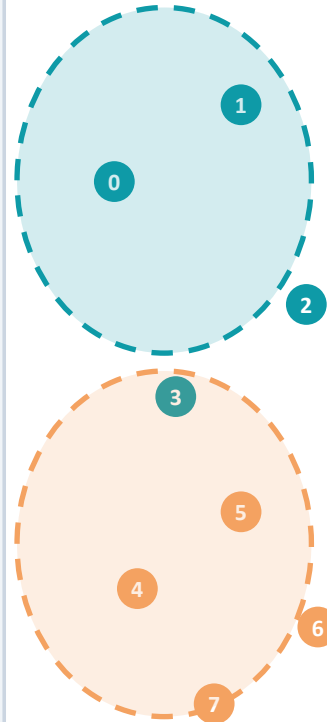
Each node in
its own community

Step 2



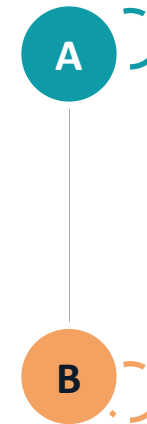
Node moves to
best neighbor community

Step 3



Phase 1 complete:
communities formed

Phase 2



Communities collapsed
to super-nodes

Iterate



Phase 1 re-runs
on super-node graph

◀ Phase 1: Modularity Optimization ▶

◀ Phase 2 ▶

◀ Repeat ▶

Complexity & Key Properties



$O(n \log n)$

Time Complexity

Nearly linear in practice

$O(n)$

Space Complexity

Only node/edge lists needed

0 to 1

Modularity Range

Higher values = clearer communities

✓ Strengths

- Fast on massive graphs (millions of nodes)
- No need to pre-specify k (number of clusters)
- Produces hierarchical community structure
- Widely implemented (NetworkX, igraph, etc.)

✗ Limitations

- Resolution limit: misses small communities in large graphs
- Non-deterministic (random initialization)
- Assumes undirected, weighted graphs by default
- Can vary across runs

Real-World Applications



Social Networks

Detect friend groups, echo chambers, and influencer communities on Twitter/Facebook

Bioinformatics

Identify protein interaction modules and gene co-expression clusters

Finance

Detect correlated stock clusters and fraud rings in transaction networks

NLP / Knowledge

Build topic clusters and entity graphs from document co-occurrence networks

Infrastructure

Partition power grids and telecom networks for resilience analysis

Recommendation

Group users with similar tastes in collaborative filtering systems

Python Implementation



Python · python-louvain (community)

```
import networkx as nx
import community as community_louvain
import matplotlib.pyplot as plt

# Create or load a graph
G = nx.karate_club_graph()

# Run Louvain clustering
partition = community_louvain.best_partition(G)

# partition = {node_id: community_id, ...}
print(f'Communities found: {max(partition.values())+1}')

# Visualize
pos = nx.spring_layout(G)
nx.draw(G, pos, node_color=list(partition.values()),
        cmap=plt.cm.tab20, with_labels=True)
plt.show()
```

Install

```
pip install python-louvain networkx
matplotlib
```

resolution param

Controls cluster granularity (default=1). Higher = smaller clusters.

randomize

Pass randomize=False for reproducible results (slower).

weighted graphs

Edge 'weight' attribute is used automatically if present.

Key Takeaways Louvain Clustering



- 01 Louvain maximizes modularity Q by greedily reassigning nodes and aggregating communities.
- 02 Two phases repeat: local optimization and network aggregation, until Q no longer improves.
- 03 The algorithm is near-linear in time — suitable for million-node graphs.
- 04 Outputs a hierarchy of communities, not a flat partition.
- 05 Widely used in social networks, biology, finance, and NLP.

What is a K-Nearest Neighbor (KNN) Graph?



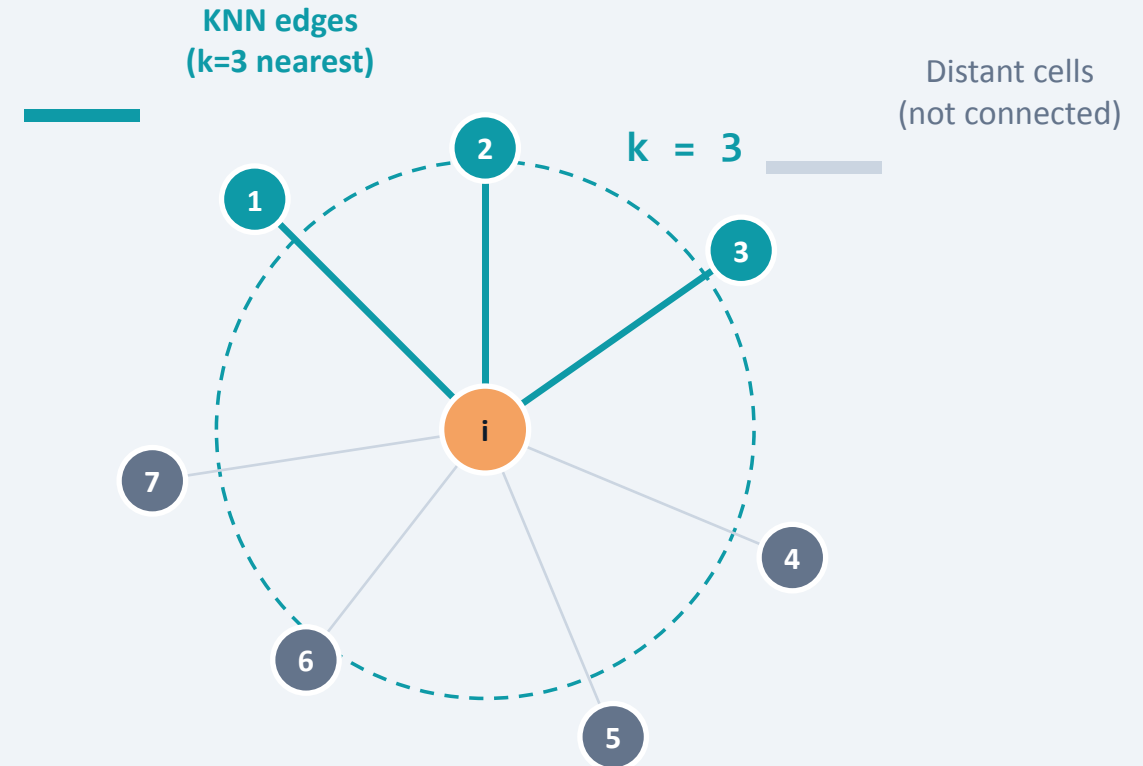
Definition

Given a set of n cells, each represented as a point in PCA space:

For every cell i , find its k most similar cells (by Euclidean distance).

Draw a directed edge from i to each of its k neighbors.

The result is the KNN graph: a sparse representation of local structure in high-dimensional expression space.



The Full Seurat Graph Pipeline



1 PCA Reduce 20,000+ genes to top N principal components (typically 10–50 PCs).

2 KNN For each cell, find k nearest neighbors in PCA space (Euclidean distance). Default k=20.

3 SNN Reweight KNN edges by Jaccard similarity of neighborhood overlap. Prune edges below threshold.

4 Louvain / Leiden Run community detection on weighted SNN graph. Resolution parameter controls granularity.

5 Clusters Each community = candidate cell type. Visualize with UMAP/tSNE; validate with marker genes.

FindNeighbors() builds steps 2–3. FindClusters() runs step 4.